

## TOPIC 4

# RTL Design Using HDL

Subtopic 4a: Verilog syntax and constructs · 4b: Designing combinational and sequential logic using HDL · 4c: Writing RTL code for basic digital circuits

- 4a · The language — modules, types, literals, operators, and the synthesisable subset
- 4b · Modelling logic — assign, always @(\*), always @(posedge clk), and what each infers
- 4c · Real circuits — MUXes, ALUs, counters, FSMs, memories, FIFOs and a working UART
- Tools · Vivado · ModelSim · Icarus + GTKWave + Verilator + Yosys · Labs L1-L5



# What This Topic Covers, and How Much Time It Really Needs

**Topic 3 gave you the hardware vocabulary.** Topic 4 is where you stop drawing gates and start writing them — and where the majority of this module's practical hours are spent. The syllabus lists 6 theory hours for this subtopic and **40 hours of RTL Design and Implementation Labs** in the practical component. This deck is sized for that reality.

| Syllabus bullet — Topic 4                                            | Covered by                 | Slides       |
|----------------------------------------------------------------------|----------------------------|--------------|
| Introduction to Verilog syntax and constructs                        | <b>4a</b>                  | 4–27         |
| Designing combinational and sequential logic using HDL               | <b>4b</b>                  | 28–49        |
| Writing RTL code for basic digital circuits                          | <b>4c</b>                  | 50–65        |
| Practical: simulating and verifying the functionality of RTL designs | <b>Verification + Labs</b> | 55–65, 75–79 |
| Practical: designing and implementing digital circuits using HDL     | <b>Tools + Labs L1–L5</b>  | 66–79        |

## Learning outcomes — by the end of this topic you can

- Read and write the synthesisable Verilog subset fluently, and say exactly what hardware each construct becomes.
- Model combinational and sequential logic correctly — no accidental latches, no blocking/non-blocking races.
- Write a self-checking testbench with a reference model, a scoreboard and randomised stimulus.
- Take a real block — a UART, a FIFO, an FSM — from specification to verified, synthesised netlist.

## How to run the session

Deliver 4a in one sitting — it is reference material and the room will not retain it all; the workbook carries the detail. Spend the real time on 4b and 4c, at the keyboard. Every design in this deck exists as verified, runnable code in Topic4\_Lab/.



# Verilog Is Not a Programming Language

**This is the single most important idea in the topic, and the one that costs beginners the most time.** Verilog looks like C. It is not C. It DESCRIBES hardware that already exists and runs continuously and in parallel — it does not instruct a processor to do things in order.

## Verilog is not a programming language — it DESCRIBES hardware

### SOFTWARE (C, Python)

- Statements run ONE AT A TIME, in order
- A variable is a location in memory
- A loop repeats over time
- 'Faster' means fewer instructions
- Compiles to instructions for a CPU

### HARDWARE (Verilog RTL)

- Every block runs ALL THE TIME, in parallel
- A signal is a wire or a flip-flop that EXISTS
- A loop UNROLLS into repeated hardware
- 'Faster' means a shorter critical path
- Synthesises to gates on silicon

**The single most useful habit in this whole topic:**

**before you write a line of Verilog, SKETCH THE HARDWARE you want. Then write the code that describes that sketch.**

### The habit that fixes almost everything

Before you write a line of Verilog, **sketch the hardware you want** — the registers, the gates between them, what feeds what. Then write the code that describes that sketch. Designers who do this produce clean RTL from the start; designers who type first and hope produce latches, races and timing failures.

## SUBTOPIC 4A

# Verilog Syntax and Constructs

The language itself — and the narrow subset of it that becomes hardware.

- Modules, ports, and the four levels of abstraction
- Values, nets and variables, literals, vectors
- Operators, and the width rules that cause silent bugs
- Procedural blocks, control flow, tasks, functions and generate
- The synthesisable subset — and what is simulation-only



# Four Levels of Description — Only Some Synthesise

**Verilog can describe hardware at four levels of detail.** All four are legal Verilog and all four simulate. Only the middle two are what you will write, and the synthesisable subset is a real, narrow boundary you must learn.

## Four levels of abstraction — all legal Verilog, only some synthesisable



### Why RTL is the level we work at

Register-transfer level describes WHAT happens to the data between clock edges, without saying which gates do it. That is abstract enough to be readable and portable between technologies, and concrete enough that the resulting hardware is predictable. Gate-level is what synthesis OUTPUTS; you should be able to read it, not write it.



# Anatomy of a Verilog Module

**The module is the only unit of design in Verilog.** It is a box with a name, a set of ports, and a body. Everything you build in this topic is a box, or a box made of boxes.

## Anatomy of a Verilog module — every design you write has this shape

### colour key

- structure
- input ports
- output ports
- internal signal
- the logic itself

```
module adder4 #(parameter W = 4) (  
    input  [W-1:0] a, b,  
    input          cin,  
    output [W-1:0] sum,  
    output          cout  
);  
  
    wire [W:0] tmp;  
  
    assign tmp  = a + b + cin;  
    assign sum  = tmp[W-1:0];  
    assign cout = tmp[W];  
  
endmodule
```

### name + parameters

sized at instantiation

### port list

the module's interface

### internal signals

not visible outside

### the design body

what it actually does

**A module is a BOX with a name and a set of ports. Everything you build in this topic is a box, or a box made of boxes.**

This is ANSI style, with ports declared in the header. The older non-ANSI style lists bare names there and declares them again below — expect it in legacy code.

### Port directions

- input** — read only inside the module.
- output** — driven by exactly ONE source.
- inout** — bidirectional; only on real chip pins.

### Two port styles

ANSI (shown): direction, type and width all in the header. Use this.

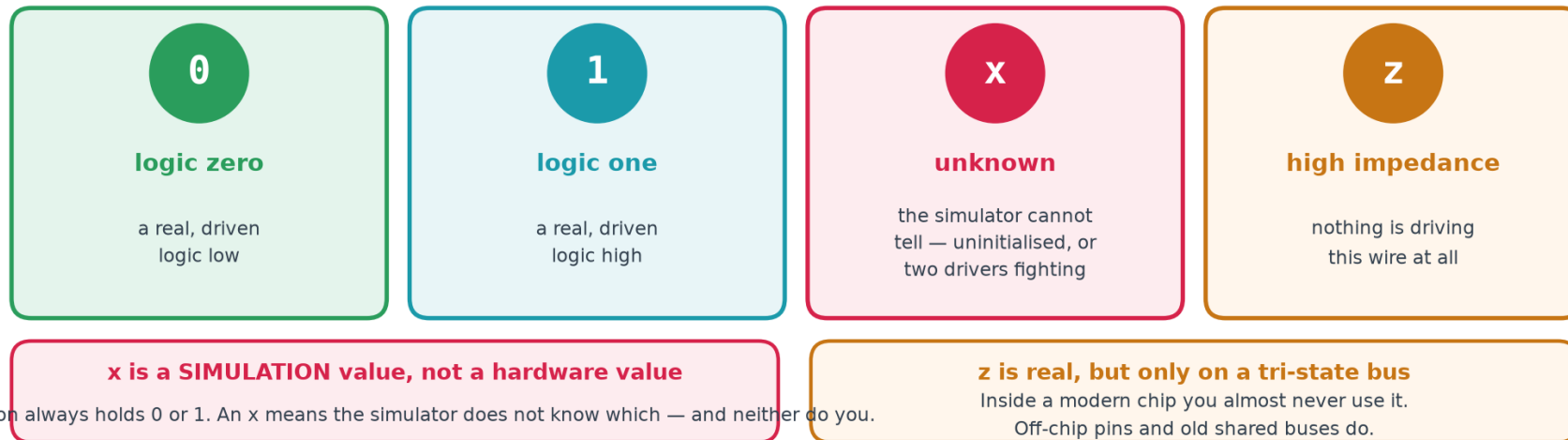
Non-ANSI: bare names in the header, declared again in the body. You will meet it in older code; do not write new code that way.



# Signals Have Four Values, Not Two

**Verilog's logic type has four states.** Two of them are hardware; two of them are things the SIMULATOR needs to say. Confusing the two categories is the root of a great deal of wasted debugging time.

## Verilog signals have FOUR values, not two



**Chasing an x back to its source is the single most common debugging task in RTL. Start at the first signal that turns x and work backwards.**

### Where an x comes from — the five usual suspects

1. A register that was never reset.
2. A wire nothing drives.
3. A wire TWO things drive (a multi-driver).
4. Reading past the end of a vector or an array.
5. Arithmetic on a value that is already x — x is contagious, which is why you must chase it back to the FIRST signal that turned x, not the one you noticed.



# wire or reg? The Rule Is About How You Assign

**This is a naming accident that has confused students since 1985.** A **reg** is NOT a register. It is a variable in the simulator. Whether it becomes a flip-flop depends entirely on HOW you assign it, and not at all on what you called it.

wire or reg? The rule is about HOW you assign, not what it becomes

wire (a NET)

- Assigned by assign
- Assigned by a module port connection
- Never assigned inside always/initial
- Models a physical connection
- The default type of an undeclared port

reg (a VARIABLE)

- Assigned inside always or initial
- Assigned with = or <=
- Never driven by assign
- Holds its value between assignments
- in the SIMULATOR. That is all it means.

The misunderstanding that costs beginners the most time

A reg is NOT a register. It is a simulator variable. Whether it becomes a flip-flop depends ENTIRELY on how you assign it:

reg in always @(\*) → combinational gates

reg in always @(posedge clk) → a flip-flop

| You want                          | Declare it         | Assign it with                  |
|-----------------------------------|--------------------|---------------------------------|
| A wire between two things         | wire               | assign, or a port connection    |
| Combinational logic in a block    | reg                | = inside always @(*)            |
| A flip-flop                       | reg                | <= inside always @(posedge clk) |
| A module output driven by assign  | wire (the default) | assign                          |
| A module output driven by a block | reg                | inside the always block         |





# Reading and Writing Number Literals

**Every constant in Verilog has a WIDTH as well as a value.** Leaving the width off does not mean 'whatever fits' — it means 32 bits, signed. That default is responsible for a whole family of silent bugs.

## Reading a Verilog number literal



| literal      | meaning                                            | value        |
|--------------|----------------------------------------------------|--------------|
| 4'b1010      | 4 bits, binary                                     | 10           |
| 8'hFF        | 8 bits, hex                                        | 255          |
| 8'd200       | 8 bits, decimal                                    | 200          |
| 8'o17        | 8 bits, octal                                      | 15           |
| 8'b1010_0101 | underscores are ignored — use them for readability | 165          |
| 4'bx         | all four bits unknown                              | xxxx         |
| 4'b10z1      | bit 1 is high-impedance                            | 10z1         |
| 42           | UNSIZED — 32 bits, signed. Avoid this.             | 32'd42       |
| -8'd3        | the MINUS applies to the whole literal             | 8'b1111_1101 |

**House rule: ALWAYS size your literals. An unsized 1 in a 64-bit expression is a 32-bit 1, and the top 32 bits silently become zero.**

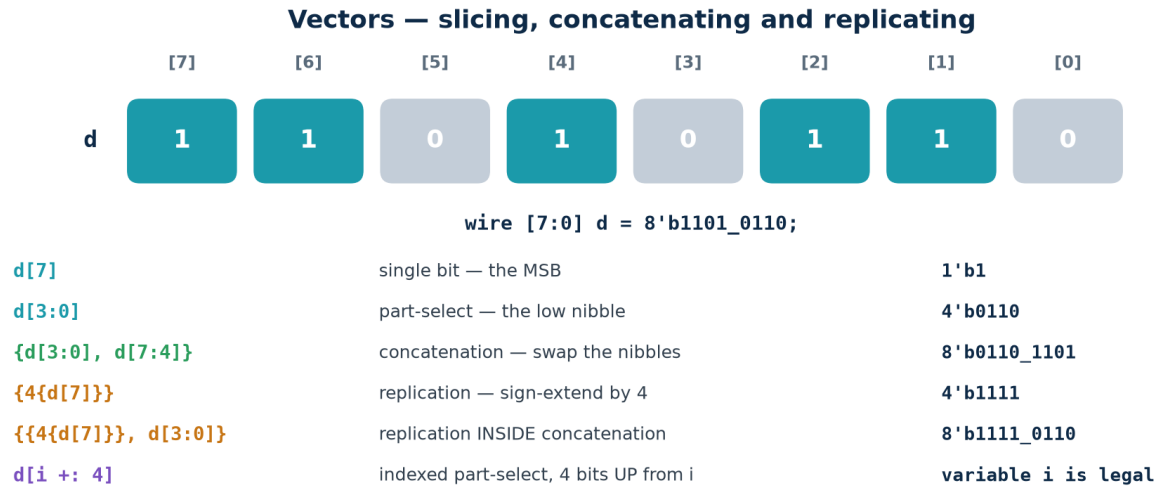
### The house rule

**Size every literal.** Write `8'd0` not `0`; `4'b0000` or `{W{1'b0}}` not a bare zero. It costs three keystrokes and removes an entire class of bug.



# Slicing, Concatenating and Replicating

**Almost every signal in real RTL is a vector — a bus.** These four operations are how you take buses apart and put them back together, and you will use them constantly.



Keep every vector declared [MSB:0]. Mixing [7:0] and [0:7] in one design is legal, confusing, and a reliable source of reversed buses.

## Arrays are NOT vectors

`reg [7:0] v;` one 8-bit vector

`reg v [0:7];` eight 1-bit elements

`reg [7:0] m [0:255];` 256 bytes — a memory

**You can slice a vector; you index an array.**

## Indexed part-select

A plain slice needs constant bounds: `d[i:j]` is illegal for variable i.

Use `d[i +: 4]` (4 bits UP from i) or `d[i -: 4]` (4 bits DOWN). The WIDTH is constant; only the position varies — which is exactly what a multiplexer can do.



# The Operators You Will Actually Use

## The Verilog operators you will actually use

|                   |                                                                                                                                            |                                                                 |                      |                                                                                                    |                                                                                                                                          |
|-------------------|--------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------|----------------------|----------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Arithmetic</b> | <code>+</code> <code>-</code> <code>*</code><br><code>/</code> <code>%</code>                                                              | add, subtract, multiply<br>in RTL, only by a power of 2         | <b>Bitwise</b>       | <code>&amp;</code> <code> </code> <code>^</code> <code>~</code> <code>~^</code>                    | bit by bit; result is a vector                                                                                                           |
| <b>Relational</b> | <code>&lt;</code> <code>&lt;=</code> <code>&gt;</code> <code>&gt;=</code>                                                                  | comparison; the result is 1 bit                                 | <b>Reduction</b>     | <code>&amp;a</code> <code> a</code> <code>^a</code>                                                | collapse a whole vector to ONE bit<br><i><code>^a</code> = parity, <code> a</code> = any bit set, <code>&amp;a</code> = all bits set</i> |
| <b>Equality</b>   | <code>==</code> <code>!=</code><br><code>===</code> <code>!==</code><br><i><code>===</code> is TESTBENCH ONLY — it does not synthesise</i> | gives x if either side has x or z<br>compares x and z literally | <b>Shift</b>         | <code>&lt;&lt;</code> <code>&gt;&gt;</code><br><code>&lt;&lt;&lt;</code> <code>&gt;&gt;&gt;</code> | logical shift, zero fill<br>arithmetic shift, sign fill                                                                                  |
| <b>Logical</b>    | <code>&amp;&amp;</code> <code>  </code> <code>!</code>                                                                                     | whole vector as true/false; 1-bit result                        | <b>Conditional</b>   | <code>?</code> <code>:</code>                                                                      | the ternary MUX — used everywhere                                                                                                        |
|                   |                                                                                                                                            |                                                                 | <b>Concatenation</b> | <code>{ }</code> <code>{n{ }}</code>                                                               | join and replicate                                                                                                                       |

**Precedence, tightest first:** `~ ! * / % + - << >> < <= > >= == != === !== & ^ | && || ? :` — **when in doubt, bracket it.**

### Logical vs bitwise — a classic slip

`a && b` treats each side as true/false and gives ONE bit.

`a & b` operates bit by bit and gives a VECTOR.

For `4'b0011` and `4'b1100`: `&&` gives 1, `&` gives 0000.

### Reduction operators are underused

`|req` — is ANY bit set? (a valid flag)

`&full` — are ALL bits set?

`^data` — parity, in one character.

`~|result` — is the result zero? That is the ALU's Z flag.



# Where Silent Bugs Come From

**Verilog will not warn you when a value does not fit.** It truncates, silently, and the simulation and the hardware agree with each other — and both are wrong. This is the most common cause of 'it worked yesterday'.

## Width and sign rules — where silent bugs come from

### The context-determined width rule

Verilog first finds the **WIDEST** operand in the whole expression — the left-hand side included — then extends every operand to that width **BEFORE** evaluating.

```
wire [3:0] a = 4'd9, b = 4'd8;  
wire [3:0] s4 = a + b;           // 17 truncates to 4'd1      WRONG  
wire [4:0] s5 = a + b;           // 5'd17                  correct  
wire [4:0] s6 = {1'b0, a} + b;    // explicit — clearest of all
```

### Signed arithmetic

A vector is **UNSIGNED** unless you declare it **signed**.  
If **ANY** operand is unsigned, the **WHOLE** expression is unsigned — so one stray signal inverts your compare.

### Two habits that prevent all of it

1. Make the result one bit wider than the widest operand.
  2. Size every literal; never rely on an unsized constant.
- Then lint it — Verilator -Wall finds width bugs for free.

### This is not hypothetical — it happened while writing this lab

The UART in Topic4\_Lab originally wrote its bit-timing limit as **CLKS\_PER\_BIT[CW-1:0]**. With **CLKS\_PER\_BIT = 16**, **CW** is 4 — and 16 does not fit in 4 bits, so it truncated to ZERO.

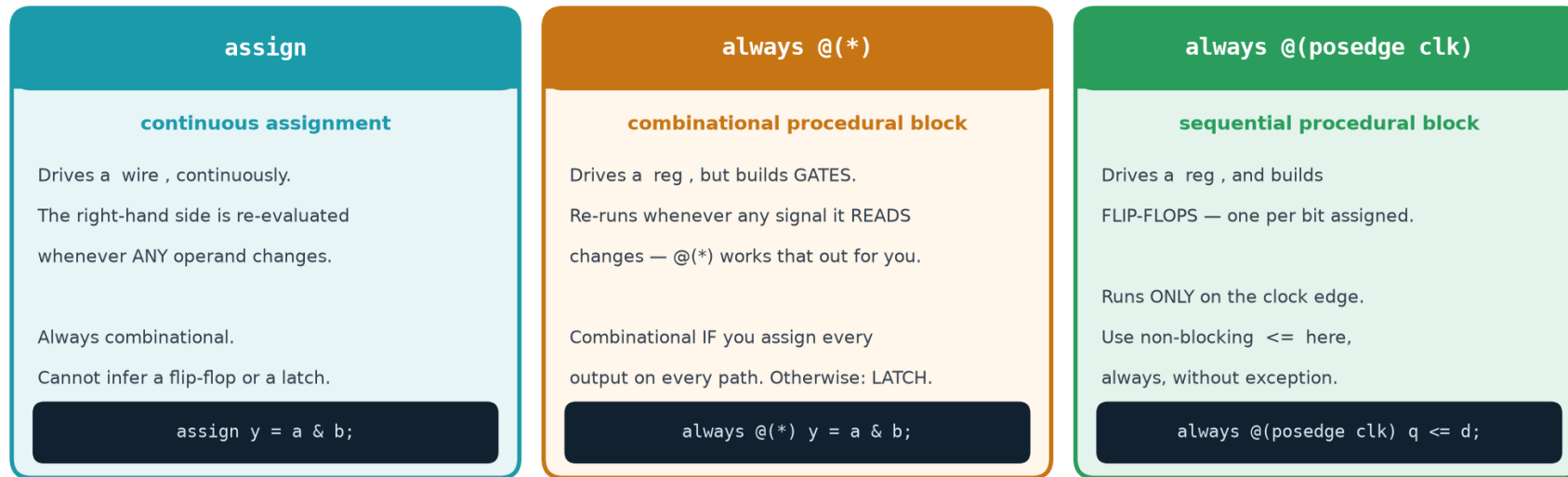
The receiver stopped waiting half a bit before sampling, began sampling on bit boundaries instead of bit centres, and corrupted *some* byte patterns but not others. 0x00 passed; 0xFF came back as 0xF7. See the UART lab for the full story.



# The Three Ways to Describe Logic

**Everything synthesisable in Verilog is written with one of three constructs.** Learn what each one BECOMES and you can already read most RTL. The rest of subtopic 4b is simply the detail of using them correctly.

## The three ways to describe logic — and what each becomes



Choosing between them is not a style question. It decides whether you get wires, gates, or registers.

| Construct             | Written as                       | Becomes            | Use it for                         |
|-----------------------|----------------------------------|--------------------|------------------------------------|
| Continuous assignment | assign y = ...;                  | A cloud of gates   | Short combinational expressions    |
| Combinational block   | always @(*) ... = ...            | A cloud of gates   | Longer logic: case, if/else chains |
| Clocked block         | always @(posedge clk) ... <= ... | Flip-flops + gates | Anything that must remember        |



# assign — Wiring, Not Instruction

**An assign statement is a permanent connection.** It is not executed once — it is ALWAYS true. Whenever anything on the right-hand side changes, the left-hand side follows, in zero simulation time. Think of it as solder, not as a line of a program.

## Continuous assignments — order-independent, always live

```
// The right-hand side is re-evaluated whenever ANY of a, b, sel, cin changes.
assign y      = a & b;           // one AND gate
assign z      = sel ? p : q;    // a 2:1 multiplexer
assign {co, sum} = a + b + cin;  // a 9-bit result split into carry and sum
assign parity  = ^data;         // an XOR tree
assign any_req = |req;          // an OR tree

// Order does not matter. These three lines describe the same circuit
// no matter which order you type them in — because they are wires.
assign c = a ^ b;
assign e = c & d;
assign d = ~a;
```

## Rules

- The left side must be a **wire** (net), never a reg.
- A wire may be driven by only ONE assign. Two drivers give x.
- No if/case/for — expressions only. Use ?: for choices.

## When to reach for it

Use assign for anything that fits comfortably on one line and has no choice structure. The moment you find yourself nesting three conditional operators, stop and rewrite it as an **always @(\*)** block with a case statement — it will be far easier to read and reviews better.



# initial and always — and What a Sensitivity List Means

**A procedural block contains statements that run in order, inside the block.** The block itself is triggered by its sensitivity list. Blocks run CONCURRENTLY with each other; statements run SEQUENTIALLY within one block.

## The forms you will meet

```
always @(*)          begin ... end    // re-runs whenever any input changes -> combinational
always @(posedge clk) begin ... end    // runs once per rising clock edge      -> flip-flops
always @(posedge clk or negedge rst_n) // adds an ASYNCHRONOUS reset

initial              begin ... end    // runs ONCE at time 0 -- SIMULATION ONLY
always               begin ... end    // runs forever -- used for clock generation in a TB
```

@(\*)

Automatic sensitivity: the tool works out every signal READ inside the block and triggers on all of them. Always use this form for combinational logic — a hand-written list such as @(a or b) is a bug waiting to happen the day someone adds a signal to the expression and forgets the list.

@(posedge clk)

Edge sensitivity: the block runs at the instant the clock rises, and at no other time. Everything assigned with <= inside becomes a flip-flop clocked by clk.

initial

Runs once at time zero. There is no hardware equivalent of 'do this once at time zero' in ASIC logic, so initial is for testbenches. (FPGA tools DO honour initial for register power-up values; ASIC tools do not — never rely on it.)



## if / else — a Priority Structure in Hardware

**An if/else chain does not 'test conditions one after another'.** It synthesises to a chain of multiplexers in which the FIRST condition has the highest priority. That chain has depth — and depth is delay.

### Priority — deep, slow, sometimes required

```
if (a) y = 0;  
else if (b) y = 1;  
else if (c) y = 2;  
else      y = 3;
```

Three multiplexers in series. Use it when the conditions really do overlap and one must win — an interrupt controller, a bus arbiter.

### Parallel — shallow, fast

```
case (sel)  
  2'd0: y = 0;   2'd1: y = 1;  
  2'd2: y = 2;   2'd3: y = 3;  
endcase
```

One multiplexer. Use it when the conditions are mutually exclusive — which they usually are.

### The rule that prevents accidental latches

**Inside always @(\*), every branch must assign every output.** If a path through the code leaves a variable unassigned, the synthesiser must build something that REMEMBERS the old value — a latch. Two ways to guarantee you never do this: (1) write a **default assignment** at the top of the block, or (2) always write a final **else / default:**. We come back to this in detail in 4b.





# case, casez, casex — and Why casex Is Banned

**case compares the selector against each label using 4-state equality.** Its two wildcard variants let labels contain don't-cares — one of them safely, one of them not.

## case and casez

```
// case -- exact match, including x and z. The workhorse.
case (opcode)
  4'b0000: y = a + b;
  4'b0001: y = a - b;
  default: y = 8'd0;      // ALWAYS write a default
endcase

// casez -- ? and z in the LABEL are don't-care. Correct for priority encoders.
casez (req)
  4'b1??? : grant = 2'd3;   // bit 3 set -- lower bits irrelevant
  4'b01?? : grant = 2'd2;
  4'b001? : grant = 2'd1;
  default: grant = 2'd0;
endcase
```

## casex — do not use it

casex also treats an **x in the SELECTOR** as a wildcard. During simulation an uninitialised signal will then match the FIRST label and appear to work, while the synthesised hardware behaves differently. It hides exactly the bug you most need to see. Most coding standards ban it outright.

## full\_case / parallel\_case

These synthesis pragmas promise the tool that all cases are covered / mutually exclusive. If the promise is false, **simulation and synthesis silently disagree** — the hardest class of bug there is. Write a real default instead; it costs nothing and cannot lie.



# for, while, repeat — Loops Are Unrolled, Not Executed

**A for loop in synthesisable Verilog is a copy-paste instruction to the tool.** It does not create a counter or take time. The tool duplicates the loop body once per iteration — so the bounds must be constants, and eight iterations means eight copies of the hardware.

## Synthesisable loop — constant bounds

```
integer i;
always @(*) begin
    cnt = 4'd0;
    for (i = 0; i < 8; i = i + 1)
        cnt = cnt + data[i];          // becomes 8 adders in a tree -- NOT a loop in hardware
end

// Reversing a bus -- 8 wires crossed over, zero gates, zero delay.
always @(*)
    for (i = 0; i < 8; i = i + 1) rev[i] = fwd[7-i];
```

## Synthesisable

**for** with constant start, limit and step.

**repeat (N)** with constant N — also unrolled.

Loop variable is integer or genvar — it does not exist in the hardware.

## Simulation only

**while**, **forever**, and any loop whose bound depends on a signal. The tool cannot know how many copies to make, so it refuses.

*If you need 'repeat until a signal changes', that is a state machine or a counter — build it explicitly.*



# Functions and Tasks

**A function is a named expression; a task is a named block of statements.** A function that obeys the rules synthesises — into a duplicate copy of its logic at every call site. Tasks are, in practice, a testbench tool.

## Function vs task

```
// FUNCTION -- synthesisable. No time control, at least one input, returns one value.
function [3:0] bcd_digit;
    input [7:0] v;
    begin
        bcd_digit = v % 10;          // the function name IS the return variable
    end
endfunction
assign d0 = bcd_digit(count); // one copy of the logic here ...
assign d1 = bcd_digit(other); // ... and a SECOND, independent copy here
// TASK -- may consume time, may have multiple outputs. Testbench use.
task send_byte(input [7:0] b);
    begin
        tx_data = b; tx_start = 1'b1; @(posedge clk); tx_start = 1'b0;
        wait (tx_busy == 1'b0);
    end
endtask
```

## The cost you must remember

**A function call is not a subroutine call — there is no 'calling' at run time. Each call site gets its own physical copy of the logic.** Four calls to a 16-bit multiplier function means four multipliers on the die. If you want ONE shared unit used at different times, that is a datapath with a multiplexer in front of it and a controller — a design decision, not a syntax choice.



# Writing One Module That Fits Every Size

**A parameter is a constant that the instantiating module can override.** Parameterising is the difference between an 8-bit counter and a counter — between code you write once and code you copy and edit five times, introducing a bug in the fourth copy.

**Parameters — write a module once, use it at every width**

```
module counter #(
    parameter W = 8
) (
    input      clk,
    input      rst_n,
    output reg [W-1:0] q
);
    always @(posedge clk or negedge rst_n)
        if (!rst_n) q <= {W{1'b0}};
        else      q <= q + 1'b1;
endmodule
```

ONE source file

```
counter #(.W(4)) u_small (...);
→ 4 flip-flops
```

```
counter #(.W(16)) u_medium(...);
→ 16 flip-flops
```

```
counter #(.W(64)) u_big (...);
→ 64 flip-flops
```

THREE different circuits

## parameter vs localparam vs `define

parameter — overridable at instantiation. Use for widths, depths, timing.

localparam — NOT overridable. Use for state names and derived constants.    `define — global text substitution; use sparingly.

## Parameterised module and an override

```
module counter #(parameter integer W = 8, parameter [W-1:0] MAX = {W{1'b1}})
    (input clk, rst_n, en, output reg [W-1:0] q, output wire tc);
    localparam [W-1:0] ZERO = {W{1'b0}};    // localparam CANNOT be overridden
    assign tc = en & (q == MAX);
    ...
endmodule

counter #(.W(4), .MAX(4'd9)) u_dig0 (.clk(clk), .rst_n(rst_n), ...);    // a decade counter
```

Use **localparam** for anything derived or internal — state encodings, computed widths — so that a careless override cannot break the module's internal consistency. Use **\$clog2(N)** to compute the number of bits needed to count to N.



# generate — Building Structure Out of a Loop

**generate lets a loop or an if create INSTANCES and blocks, not just logic.** It runs at elaboration time — before simulation starts — and its result is a fixed structure. This is how you build an N-bit ripple adder from one full-adder module.

## generate — writing N copies of something without writing them out

```
genvar i;
generate
  for (i = 0; i < W; i = i + 1) begin : bit_slice
    full_adder u_fa (
      .a (a[i]),
      .b (b[i]),
      .cin (c[i]),
      .sum (sum[i]),
      .cout(c[i+1])
    );
  end
endgenerate
```

elaborates to



four real, separate instances

### generate is ELABORATION-time, not run-time

The loop runs once, while the tool is building the design. Nothing 'iterates' in hardware.

The : bit\_slice label is not optional decoration — it names the instances, and you need those names for timing reports and waveforms.

### generate for — Topic4\_Lab/rtl/adder\_gen.v

```
genvar i;
generate
  for (i = 0; i < W; i = i + 1) begin : bit_slice    // the label is REQUIRED
    full_adder u_fa (.a(a[i]), .b(b[i]), .cin(c[i]), .sum(sum[i]), .cout(c[i+1]));
  end
endgenerate
// Instances are named bit_slice[0].u_fa, bit_slice[1].u_fa, ... -- use these in $dumpvars
```

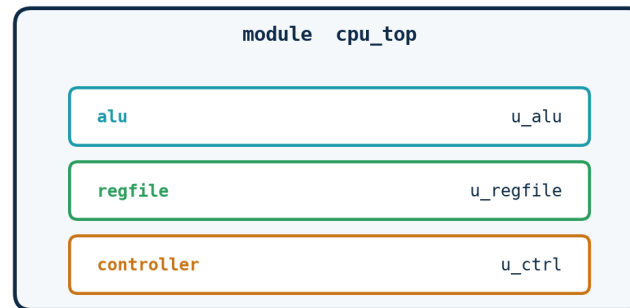
**generate if** selects between whole implementations at elaboration — for example a fast carry-lookahead adder when W is large and a small ripple adder when it is not. The unselected branch is not elaborated at all, so it need not even be legal for the chosen parameters.



# Instantiating Modules — Always By Name

**Real designs are trees of modules.** A module is a TYPE; an instance is a physical copy. Two instances of the same module are two separate pieces of hardware that share nothing.

## Hierarchy — building a design out of instantiated modules



one MODULE can be instantiated many times;  
each INSTANCE has its own name

```
alu #(
    .W (32)
) u_alu (
    .clk (clk),
    .a (rs1_data),
    .b (rs2_data),
    .op (alu_op),
    .result(alu_out)
);
```

NAMED port connection — always use this. Positional  
order is legal, and silently wrong the day someone adds a port.

### By name — do this

```
uart_tx u_tx (
    .clk (clk),
    .rst_n (rst_n),
    .data (tx_data),
    .tx (serial_out));
```

Order independent, self documenting, and a typo is a compile error.

### By position — do not

```
uart_tx u_tx (clk, rst_n,
              tx_data, serial_out);
```

Add a port to the module and every instance silently connects the wrong wires.  
Legal, common in old code, and a permanent source of accidents.

Naming convention used throughout this course: instances are prefixed **u\_**; an unconnected output is written **.name()** explicitly so a reader can see it was deliberate.



# Compiler Directives — the Backtick Family

**Directives are processed before compilation, like C's preprocessor.** They begin with a backtick, not a hash, and they are NOT terminated with a semicolon.

| Directive                              | What it does                                 | Note                                       |
|----------------------------------------|----------------------------------------------|--------------------------------------------|
| <code>`timescale 1ns/1ps</code>        | Sets the unit and precision of delays        | First line of every file; unit / precision |
| <code>`define NAME value</code>        | Text macro, referenced as <code>`NAME</code> | GLOBAL — prefix to avoid clashes           |
| <code>`include "file.vh"</code>        | Textual inclusion                            | Use for shared parameter headers           |
| <code>`ifdef / `ifndef / `endif</code> | Conditional compilation                      | Simulation-only code, ASIC vs FPGA         |
| <code>`default_nettype none</code>     | Turns off implicit wire creation             | Put it in EVERY file. See below.           |
| <code>`resetall</code>                 | Restores defaults                            | Pair with the line above at end of file    |

## ``default_nettype none` — the single most valuable line in the file

By default, a name you never declared becomes a 1-bit wire, silently. Misspell `data_valid` as `data_vaild` in a port connection and Verilog creates a new, undriven 1-bit wire and says nothing; your design gets an x and you spend an afternoon on it.

Put ``default_nettype none` at the top of every file and that typo becomes a compile error on the exact line. Every file in Topic4\_Lab does this.



# The Dollar Family — Your Simulation Instruments

**System tasks are built-in routines the SIMULATOR provides.** None of them synthesise — they are how you observe and control a simulation. Two of them (`$clog2` and `$bits`) are exceptions that are legal in synthesisable code because they are evaluated at elaboration.

| Task                                                     | Purpose                             | Typical use                                       |
|----------------------------------------------------------|-------------------------------------|---------------------------------------------------|
| <b><code>\$display</code> / <code>\$write</code></b>     | Print once, when reached            | <code>\$display("%0t got %h", \$time, q);</code>  |
| <b><code>\$monitor</code></b>                            | Print whenever any argument changes | One per simulation, in initial                    |
| <b><code>\$time</code> / <code>\$realtime</code></b>     | Current simulation time             | Timestamping messages                             |
| <b><code>\$dumpfile</code> / <code>\$dumpvars</code></b> | Write a VCD waveform file           | <code>\$dumpvars(0, tb);</code> — 0 = all levels  |
| <b><code>\$finish</code> / <code>\$stop</code></b>       | End / pause the simulation          | <code>\$finish</code> at the end of your stimulus |
| <b><code>\$random</code> / <code>\$urandom</code></b>    | Pseudo-random stimulus              | Seed it so failures reproduce                     |
| <b><code>\$fatal</code> / <code>\$error</code></b>       | Report and optionally abort         | Self-checking testbenches                         |
| <b><code>\$clog2(N)</code></b>                           | Ceiling log2 — SYNTHESISABLE        | <code>localparam CW = \$clog2(DEPTH);</code>      |

Format specifiers: `%b` `%o` `%d` `%h` for radix, `%s` string, `%t` time. Prefix with 0 — `%0d` — to drop the padding that otherwise makes output unreadable.





# What Synthesises, and What Does Not

**This is the boundary that separates a design from a testbench.** Verilog is a simulation language that HAPPENS to have a synthesisable subset. If you write something outside it, the simulation may run perfectly and the synthesiser will either reject it or — worse — quietly ignore it.

## The synthesisable subset — what becomes hardware and what does not

| SYNTHESISABLE                                                                                                                                                                                                                                                                                                                                                                    | SIMULATION ONLY                                                                                                                                                                                                                                                                                                                        |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <ul style="list-style-type: none"><li>· module / endmodule, ports</li><li>· wire, reg, parameter, localparam</li><li>· assign</li><li>· always @(*) and @(posedge clk)</li><li>· if / else, case / casez</li><li>· for loops with CONSTANT bounds</li><li>· function</li><li>· generate / genvar</li><li>· all operators except / and %<br/>(unless by a power of two)</li></ul> | <ul style="list-style-type: none"><li>· Initial blocks</li><li>· #delays ( #10 )</li><li>· wait, fork / join</li><li>· while, forever, repeat</li><li>· task with timing</li><li>· \$display, \$monitor, \$finish</li><li>· \$random, \$readmemh</li><li>· real, time, event</li><li>· === and !==</li><li>· force / release</li></ul> |

Simulation-only constructs are not bad — you need them in every testbench. They just must never appear in a design module.

### The four that catch people out

- **Delays.** #5 is ignored by synthesis. Timing comes from the clock and the technology, never from your source.
- **initial blocks.** Reset your registers with a reset signal.

### ...continued

- **Multiple always blocks driving one signal.** Illegal. One signal, one driver, one block.
- **Mixing = and <= for the same variable.** Legal Verilog, undefined behaviour in practice, rejected by every lint tool.



# House Rules — The Checklist Used in This Course

**Every rule below exists because breaking it has cost somebody a day.** They are the standing lint rules for all labs in Topic 4.

- One file, one module, and the file is named after the module.
- ``default_nettype none` at the top; ``resetall` at the bottom.
- ANSI port headers. Instantiate by name. Prefix instances with `u_`.
- Active-low signals end in `_n` (`rst_n`). Clocks are `clk`.
- Size every literal. Never write a bare 0 or 1 into a vector.
- Combinational: `always @(*)` with `=` and a default assignment. Sequential: `always @(posedge clk)` with `<=` only.
- Every case has a **default**. Never `casex`. Never `full_case/parallel_case`.
- One driver per signal. Reset every state-holding register.
- Lint with Verilator before you simulate; simulate before you synthesise; check the synthesis log for inferred latches every single time.

## Why the log matters more than the waveform

A latch warning in the synthesis log is telling you the hardware does not match your intent. The simulation will often pass anyway, because in simulation the latch happens to hold the value you expected. The mismatch appears on silicon. Read the log.



## Before We Move On — Ten Questions

**Ask the room. Do not move to 4b until most of these come back quickly.** Answers are in the workbook, section T4-A.

### Questions 1-5

1. Is **reg** a register? When is it, and when is it not?
2. What width and sign does the literal **12** have?
3. What does **{3{2'b10}}** evaluate to, and how wide is it?
4. Give one bit of output: **^8'b1101\_0110 = ?**
5. Why is **a && b** not the same as **a & b**?

### Questions 6-10

6. How many adders does an unrolled 8-iteration for loop create?
7. Name two things that make code un-synthesisable.
8. What is the difference between **parameter** and **localparam**?
9. Why does every file start with **`default\_nettype none**?
10. Why is casex banned?

### If question 1 or 5 gave the room trouble

Go back to slides 11 and 16 now rather than pressing on. Everything in 4b is built on knowing what a reg becomes and how wide an expression is; a class that is shaky on those two will not follow the latch discussion at all.

## SUBTOPIC 4B

# Designing Combinational and Sequential Logic Using HDL

From here on, every construct is judged by one question: what hardware does it produce?

- The inference map — code pattern in, hardware out
- Combinational modelling, and the latch you must never infer
- Sequential modelling, blocking vs non-blocking, and the event queue
- Reset strategy, counters, shift registers, edge detection, CDC
- State machines: styles, encodings, safe defaults
- Memories, and pipelining for speed



# The Inference Map — Learn This Table Cold

**Synthesis is pattern matching.** The tool reads your code, recognises a pattern, and drops in the corresponding hardware. There is no intelligence in it — which is good news, because it means the mapping is completely predictable once you know the patterns.

The inference map — what each RTL construct becomes in silicon

| you write                                  |   | the tool builds                             |
|--------------------------------------------|---|---------------------------------------------|
| assign y = a & b;                          | ■ | an AND gate                                 |
| assign y = s ? a : b;                      | ■ | a 2:1 multiplexer                           |
| assign y = a + b;                          | ■ | an adder — the tool picks the structure     |
| assign y = a * b;                          | ■ | a MULTIPLIER — large. Check you meant it.   |
| assign y = a / b;                          | ■ | a DIVIDER — huge, or a synthesis error      |
| always @(*) with full assignment           | ■ | combinational gates                         |
| always @(*) with a missing branch          | ■ | a transparent LATCH — a bug                 |
| case (sel) ... endcase                     | ■ | a multiplexer, one input per branch         |
| if / else if / else if ...                 | ■ | a PRIORITY multiplexer chain — slower       |
| always @(posedge clk) q <= d;              | ■ | one D flip-flop per bit of q                |
| always @(posedge clk) if (en) q <= d;      | ■ | a flip-flop with a clock enable             |
| reg [7:0] mem [0:255]; with a clocked read | ■ | a block RAM on FPGA, a memory macro on ASIC |
| for (i=0; i<N; i=i+1) with constant N      | ■ | N copies of the loop body, UNROLLED         |
| x <= y; in two different always blocks     | ■ | a multi-driver error                        |

Green = exactly what you meant. Amber = check you meant it. Red = a bug. There is no fourth category.

## The single question to ask of every line you write

**"What does this become?"** If you cannot answer that for a line of your own RTL, you have written something you do not understand, and the tool will make its own choice. Nine times out of ten that choice is a latch, a huge multiplexer, or a critical path you did not expect.



# Three Ways to Write the Same Multiplexer

**All three of these synthesise to identical hardware.** Choose by readability, not by any belief about efficiency — the synthesiser flattens all of them to the same boolean function before it optimises.

## Topic4\_Lab/rtl/mux2.v uses style 1

```
// Style 1 -- continuous assignment. Best for short expressions.
assign y = sel ? b : a;

// Style 2 -- combinational always block with if/else.
always @(*) begin
    if (sel) y = b;
    else    y = a;          // the else is what stops a latch
end

// Style 3 -- combinational always block with case. Scales best.
always @(*) begin
    case (sel)
        1'b0:    y = a;
        default: y = b;          // default covers 1'b1, 1'bx and 1'bz
    endcase
end
```

Style 1 for one-liners. Style 3 the moment there are three or more choices — a 4:1 multiplexer written as nested ternaries is unreadable, and unreadable RTL is where bugs live.



# The Accidental Latch — The Number One Beginner Bug

If a combinational block does not assign a variable on **EVERY** possible path, the variable must keep its old value when that path is taken. Keeping an old value is memory, and the only memory the tool can build from a level-sensitive block is a transparent latch.

## The inferred latch — the most common beginner bug in RTL

### WRONG — incomplete assignment

```
always @(*) begin
    if (sel)
        y = a;
end
```

**sel = 0 → y is not assigned → 'hold the old y'**  
→ the only thing that can hold a value is a LATCH

### RIGHT — default first

```
always @(*) begin
    y = 1'b0;    // default
    if (sel)
        y = a;
end
```

every path assigns y → pure gates, a 2:1 MUX

### Three rules that make an accidental latch impossible

1. Assign every output a DEFAULT value at the top of the block.
2. Give every case a default: branch — even if it is unreachable.
3. Never write a bare if without an else in a combinational block.

### How to catch one

Yosys prints `$_DLATCH_` cells in stat. Vivado logs '[Synth 8-327] inferring latch for variable y'.  
ModelSim and Verilator warn too. Treat EVERY such message as an error, never a warning.

### Why a latch is bad news

- It is transparent while enabled — data flows straight through, so it does not isolate one cycle from the next.
- Static timing analysis of latch-based paths is far harder; many flows simply cannot close timing on them.
- **It was almost certainly not what you meant.**

### The two cures — use both

- 1. Default first.** Assign every output at the top of the block, then override in the branches.
- 2. Complete every branch.** Every if has an else; every case has a default. Then grep the synthesis log for the word 'latch' before you go home.



## Seeing the Latch in a Real Tool Report

**Do not take this on trust — make the tool show you.** Topic4\_Lab/rtl/broken\_examples.v contains deliberately wrong modules so you can watch each failure appear in a synthesis report and then watch it disappear when you fix it.

### rtl/broken\_examples.v

```
// bad_latch -- BOTH always blocks are incomplete
always @(*) begin
    if (enable) y = d[0];           // no else           -> LATCH on y
end
always @(*) begin
    case (sel)
        2'b00: w = d[0];
        2'b01: w = d[1];
        2'b10: w = d[2];           // 2'b11 missing, no default -> LATCH on w
    endcase
end
```

### Verified output — run this yourself in Lab L1

```
$ yosys -p "read_verilog rtl/broken_examples.v; synth -top bad_latch; stat"
   _DLATCH_N_      1
   _DLATCH_P_      1          <-- two latches, exactly as predicted
```

*Now add the else and the default, re-run, and the two DLATCH lines vanish. That before-and-after is the exercise; the slide is only the summary.*

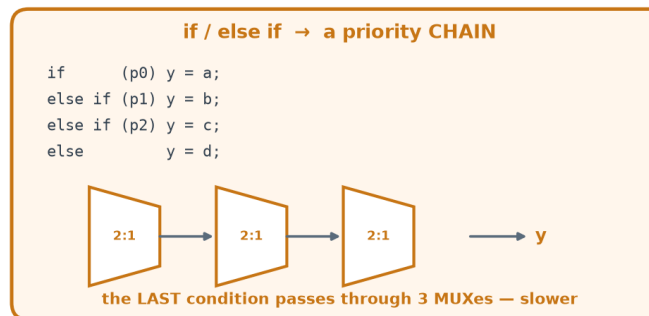
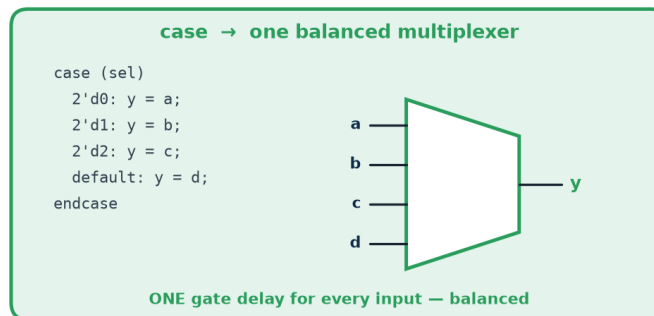




# case Becomes a Multiplexer — or a Decoder, or an Encoder

**One construct, three familiar building blocks.** Which one you get depends only on what is being selected: data (multiplexer), a one-hot output (decoder), or a priority (encoder).

**case builds a MUX; if/else-if builds a PRIORITY chain**



## When does it matter?

Use `case` when the conditions are mutually exclusive — a decoder, an opcode, an FSM. Use `if/else if` when you genuinely WANT priority: an interrupt arbiter, a bus grant. Writing `if/else-if` for a mutually exclusive choice costs you delay for nothing.

## Decoder — Topic4\_Lab/rtl/decoder3to8.v

```
always @(*) begin
  y = 8'b0;
  if (en) y[sel] = 1'b1;
end
```

A variable INDEX on the left of an assignment is a decoder. Perfectly legal, perfectly synthesisable.

## Priority encoder — rtl/priority\_encoder8.v

```
casez (req)
  8'b1??????? : {v,y} = {1'b1,3'd7};
  8'b01??????? : {v,y} = {1'b1,3'd6};
  ... default: {v,y} = {1'b0,3'd0};
casez, never casex. The valid bit tells you 'no request' apart from 'request 0'.
```



## An 8-Operation ALU With Real Flags

**This is the first design in the lab that is big enough to be interesting.** Note the defaults at the top of the block, the width-(W+1) sum that captures the carry, and the fact that signed overflow is NOT the same thing as carry.

### Topic4\_Lab/rtl/alu.v (extract)

```
wire [W:0] sum_ext = {1'b0, a} + {1'b0, b}; // ONE bit wider -> carry is free
always @(*) begin
    result = {W{1'b0}}; carry = 1'b0; overflow = 1'b0; // defaults -> no latch
    case (op)
        OP_ADD: begin
            result = sum_ext[W-1:0];
            carry = sum_ext[W];
            overflow = (a[W-1] == b[W-1]) && (result[W-1] != a[W-1]);
        end
        OP_SLT: result = ($signed(a) < $signed(b)) ? {{(W-1){1'b0}}, 1'b1} : {W{1'b0}};
        ...
        default: begin result = {W{1'b0}}; carry = 1'b0; overflow = 1'b0; end
    endcase
end
assign zero = ~|result; // reduction NOR -- one operator, a whole NOR tree
assign negative = result[W-1];
```

### The flag that always catches people

**Carry is UNSIGNED overflow; V is SIGNED overflow.** Adding 8'h7F + 8'h01 gives 8'h80: no carry out at all, but a signed overflow (127 + 1 became -128). Adding 8'hFF + 8'h01 gives 8'h00: carry out set, but no signed overflow (-1 + 1 = 0 is correct). The lab testbench checks both.



# The Clocked Block — One Template, Used Everywhere

**Almost every sequential block you will ever write has this shape.** Learn it as a pattern and type it from muscle memory; then all your attention goes on the logic instead of the boilerplate.

## The four sequential templates you will use every day

|                                                                                                                                                         |                                                                                                          |
|---------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------|
| <b>Plain register</b><br><small>no reset — pipeline stages only</small>                                                                                 | <pre>always @(posedge clk)   q &lt;= d;</pre>                                                            |
| <b>Synchronous reset</b><br><small>reset is one more input to the D logic</small>                                                                       | <pre>always @(posedge clk)   if (!rst_n) q &lt;= 4'd0;   else      q &lt;= d;</pre>                      |
| <b>Asynchronous reset</b><br><small>works with no clock — the ASIC choice</small>                                                                       | <pre>always @(posedge clk or negedge rst_n)   if (!rst_n) q &lt;= 4'd0;   else      q &lt;= d;</pre>     |
| <b>Async reset + clock enable</b><br><small>THE workhorse — use this by default</small>                                                                 | <pre>always @(posedge clk or negedge rst_n)   if (!rst_n)  q &lt;= 4'd0;   else if (en) q &lt;= d;</pre> |
| <b>Two things that are ALWAYS wrong in a clocked block</b><br>Anything except clk and an async reset in the sensitivity list.    Using = instead of <=. |                                                                                                          |

## The template — priority runs top to bottom

```
always @(posedge clk or negedge rst_n) begin
  if (!rst_n)      q <= RESET_VALUE;      // asynchronous, active-low reset
  else if (clr)    q <= {W{1'b0}};        // synchronous clear
  else if (load)   q <= din;               // priority order matters:
  else if (en)     q <= next_value;        //   reset > clear > load > enable
end
```

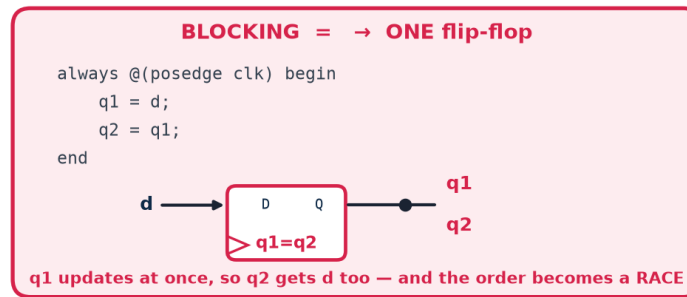
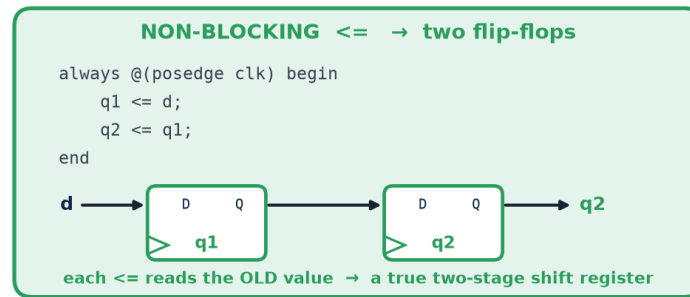
Everything assigned in this block becomes a flip-flop. Nothing else in the design may assign **q** — one signal, one driving block, no exceptions.



# Blocking and Non-Blocking — The Rule and the Reason

**The rule is two lines long and you must never break it.** Inside always @(\*) use **=**. Inside always @(posedge clk) use **<=**. Never mix the two for the same variable, and never use = in a clocked block.

## Blocking vs non-blocking — same code, different hardware



### The rule — no exceptions, ever

**always @(posedge clk)**  
**use <= (non-blocking)**

**always @(\*)**  
**use = (blocking)**

**Never mix the two in one block. Never assign the same signal from two blocks.**

Why it works: non-blocking assignments sample every right-hand side FIRST, then update together at the end of the time step — which is exactly what a bank of flip-flops does on a clock edge.

### = blocking — happens NOW

The right-hand side is evaluated and the left-hand side updated immediately, before the next statement runs. In a clocked block that makes the result depend on the ORDER you typed the statements — which no real flip-flop cares about.

### <= non-blocking — happens at the END

Every right-hand side in the block is sampled first, using the OLD values; only then are all the left-hand sides updated together. That is exactly what a bank of flip-flops does at a clock edge, which is why it is the correct operator.



## Two Flip-Flops or One? The Tool Decides

**A three-line difference that changes the hardware.** This is `bad_blocking` from the lab. Read the two versions, predict the flip-flop count, then run the synthesiser and see.

### WRONG — blocking in a clocked block

```
always @(posedge clk) begin
```

```
    q1 = d;
```

```
    q2 = q1;    // q1 is ALREADY d
```

```
end
```

q2 follows d in the same cycle. Synthesis merges them: **1 flip-flop**.

### RIGHT — non-blocking

```
always @(posedge clk) begin
```

```
    q1 <= d;
```

```
    q2 <= q1;    // the OLD q1
```

```
end
```

A real two-stage shift register: **2 flip-flops**, d delayed by two cycles.

```
$ yosys -p "read_verilog rtl/broken_examples.v; synth -top bad_blocking; stat"
$ _DFF_P_      1          <-- ONE flip-flop, not two. Verified.
```

*Change the two = to <= and the same command reports 2. Nothing else in the file changes. That is how directly the operator maps to silicon.*

### Why the tool is entitled to do that

Blocking assignment says 'update now, before the next statement'. So at the clock edge q1 takes d, and the very next statement gives q2 the NEW q1 — which is d. The tool sees two registers holding the same value and keeps one. It is not being clever; it is obeying the semantics you wrote.

### And if you swap the two lines?

Write **q2 = q1; q1 = d;** and you get two flip-flops and correct behaviour — from the same operator. That is the real objection to blocking assignment in a clocked block: the hardware depends on the ORDER you typed the lines, which is not a property real flip-flops have.

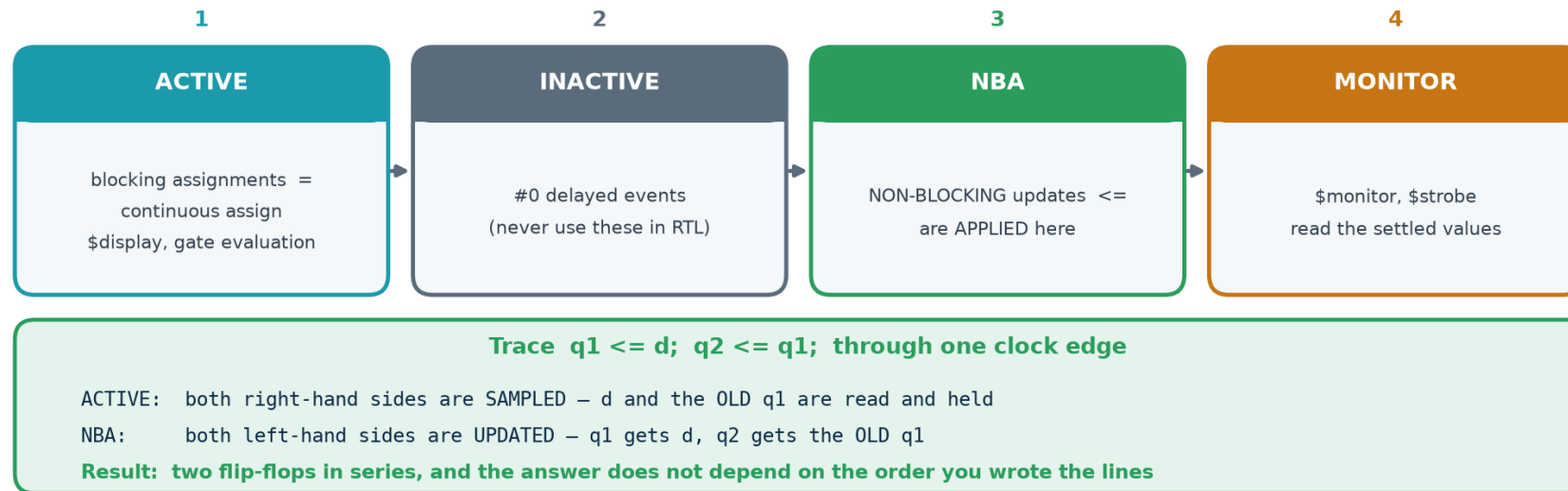


# The Stratified Event Queue — Why $\leq$ Actually Works

**This is the mechanism, for the students who want to know WHY.** At a single simulation time step the simulator processes events in defined regions, in order. Non-blocking updates land in a later region than the right-hand-side evaluations — and that separation is the whole trick.

## Why the rule works — Verilog's event queue for one time step

*the simulator empties region 1 completely, THEN region 2, and so on — all at the same simulation time*



### What this buys you

Every clocked block in the design samples its inputs in the Active region using the values that existed BEFORE the edge, and every register updates in the NBA region afterwards. So no clocked block can ever see another block's new value within the same edge — which is precisely the behaviour of real flip-flops, and why the order in which the simulator happens to visit your always blocks cannot change the answer. Use  $=$  in a clocked block and you throw that guarantee away.



# Synchronous or Asynchronous Reset — Pick One and Be Consistent

**Reset is a design decision, not a detail.** Mixing styles inside one design is how you get a block that comes out of reset two cycles after its neighbour and starts in an impossible state.

## Asynchronous reset (used in this lab)

```
always @(posedge clk or negedge rst_n)
    if (!rst_n) q <= 1'b0;
    else      q <= d;
```

+ Works with no clock running — safe at power-up.

– Release must be synchronised, or flip-flops can come out of reset on different cycles (and can go metastable).

## Synchronous reset

```
always @(posedge clk)
    if (!rst_n) q <= 1'b0;
    else      q <= d;
```

+ One clock domain, easy timing, no release problem.

– Needs a running clock; costs logic in front of the D input; a glitch shorter than a cycle is missed.

## Reset-release synchroniser — assert asynchronously, release synchronously

The standard solution: drive the reset tree from two flip-flops clocked by clk whose asynchronous reset is the raw input. The reset asserts the instant the signal falls, and releases cleanly on a clock edge two cycles later. Every serious design does this; it costs two flip-flops.



# A Counter Is a Register Plus an Incrementer

**Counters are the workhorse of sequential design — timers, address generators, dividers, bit counters in a UART.** The lab counter is parameterised, up/down, loadable and has a terminal count.

## Topic4\_Lab/rtl/counter.v — W=4, MAX=15

```
assign tc = en & (up ? (q == MAX[W-1:0]) : (q == {W{1'b0}}));

always @(posedge clk or negedge rst_n) begin
    if (!rst_n)      q <= {W{1'b0}};
    else if (load)   q <= din;
    else if (en) begin
        if (up)      q <= (q == MAX[W-1:0]) ? {W{1'b0}} : q + 1'b1;
        else         q <= (q == {W{1'b0}}) ? MAX[W-1:0] : q - 1'b1;
    end
end
```

```
$ yosys -p "read_verilog rtl/counter.v; synth -top counter; stat"
Number of cells: 38      $_DFF_NP0_ 4      <-- 4 bits of state, as expected
```

Set MAX = 9 and you have a BCD decade counter with no other change — that is what parameterisation buys. Chain two of them with **tc** as the enable of the next and you have a 0–99 counter.





# Shift Register and Edge Detector

**Two tiny modules that appear in almost every design you will ever build.** The edge detector in particular is the standard way to turn a level into a single-cycle pulse.

## Shift register — the concatenation idiom

```
always @(posedge clk or negedge rst_n)
    if (!rst_n) q <= {W{1'b0}};
    else if (en) q <= {q[W-2:0], sin};
```

One line shifts the whole register left and inserts the new bit. Reverse the concatenation for a right shift. This idiom builds serialisers, deserialisers, CRC engines and LFSRs.

## Edge detector — one register and a gate

```
always @(posedge clk or negedge rst_n)
    if (!rst_n) sig_d <= 1'b0; else sig_d <= sig;
```

```
assign rise = sig & ~sig_d;
```

```
assign fall = ~sig & sig_d;
```

Remember what the signal was last cycle; compare. rise is high for exactly one clock cycle.

## A testbench lesson from building this lab

**The first edge-detector test failed. The cause was not the design — the testbench changed the input in the middle of a clock cycle, so the pulse was half a cycle wide and the check sampled it at the wrong moment.** Drive your stimulus just AFTER the active clock edge, and sample just BEFORE the next one. Half of all 'the design is broken' reports are really this.



# Metastability and the Two-Flop Synchroniser

**A flip-flop needs its D input stable for a setup time before the edge and a hold time after it.** A signal from another clock domain cannot honour that, so the flip-flop can enter a metastable state — an output that is neither 0 nor 1 and settles at an unpredictable moment. You cannot prevent this. You can only give it time to decay.

## Topic4\_Lab/rtl/synchroniser.v

```
reg [1:0] sync;
always @(posedge clk or negedge rst_n) begin
    if (!rst_n) sync <= 2'b00;
    else      sync <= {sync[0], async_in}; // stage 0 may go metastable ...
end
assign sync_out = sync[1];                // ... stage 1 has a whole cycle to settle
```

### Rules for crossing a clock domain

- Two flip-flops minimum for a single-bit LEVEL. Three at very high clock rates.
- **Never synchronise a multi-bit bus bit by bit — the bits will arrive on different cycles and you will read a value that never existed.**

### What to do instead for buses

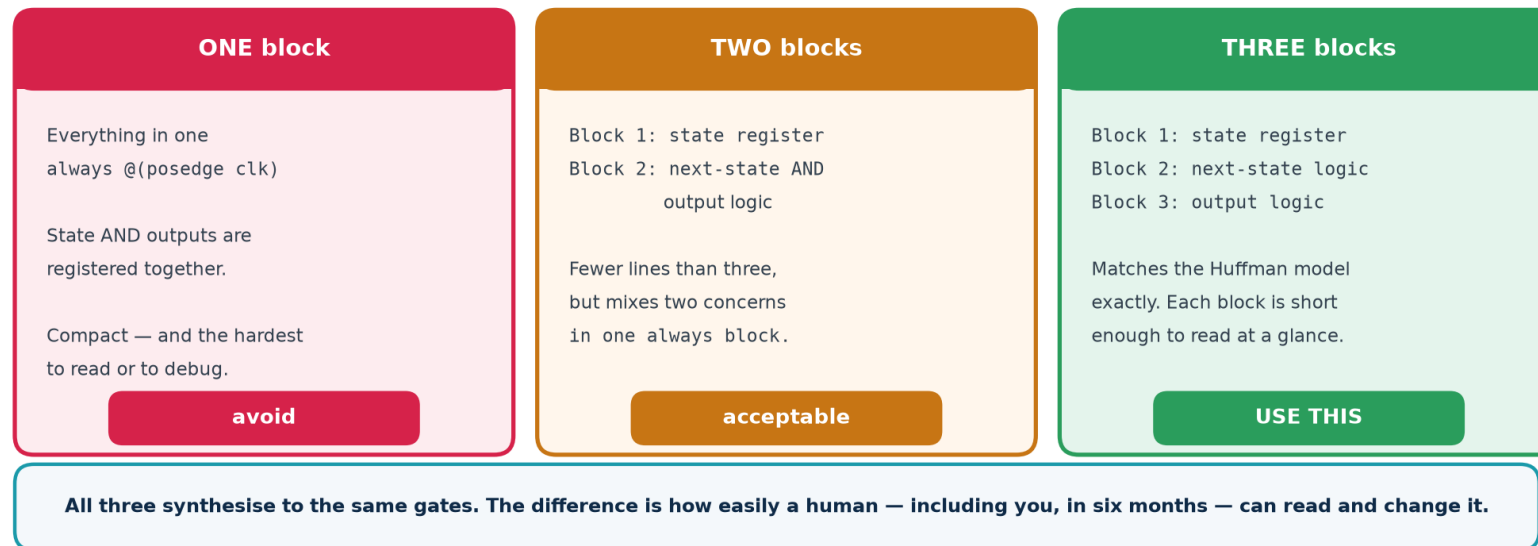
- Gray-coded pointers (that is how an asynchronous FIFO works — only one bit changes per step, so a mis-sample gives the previous value, which is safe).
- Or a request/acknowledge handshake with the data held stable throughout.



# Three Coding Styles — Use the Three-Block Style

**A finite state machine is a state register, next-state logic and output logic.** How many always blocks you split those across is a style choice — and it matters more than students expect.

## Three FSM coding styles — and which one to standardise on



| Style       | Blocks             | Pros                                          | Cons                                              |
|-------------|--------------------|-----------------------------------------------|---------------------------------------------------|
| One-block   | 1 clocked          | Compact; outputs are registered automatically | Next-state and output logic tangled; hard to read |
| Two-block   | 1 clocked + 1 comb | Clear separation of state and logic           | Outputs still mixed into the combinational block  |
| Three-block | 1 clocked + 2 comb | State, next-state and outputs each separate   | Slightly more typing — that is the whole cost     |



# A Traffic-Light Controller, Three Blocks

## Topic4\_Lab/rtl/traffic\_fsm.v — verified: 52 cells, 12 flip-flops

```
// ---- BLOCK 1 : state register (and the dwell timer) ----
always @(posedge clk or negedge rst_n) begin
    if (!rst_n)      begin state <= MAIN_GREEN; timer <= 8'd0; end
    else if (tick)   begin
        if (done)    begin state <= next; timer <= 8'd0; end
        else        timer <= timer + 1'b1;
    end
end
end
// ---- BLOCK 2 : next-state logic (pure combinational) ----
always @(*) begin
    next = state;           // default -> no latch, and a safe self-loop
    case (state)
        MAIN_GREEN : next = MAIN_YELLOW;
        MAIN_YELLOW: next = SIDE_GREEN;
        SIDE_GREEN  : next = SIDE_YELLOW;
        SIDE_YELLOW : next = MAIN_GREEN;
        default     : next = MAIN_GREEN; // SAFE FSM -- recover from an illegal state
    endcase
end
// ---- BLOCK 3 : output logic (Moore -- depends on state ONLY) ----
always @(*) begin
    main_light = RED; side_light = RED;           // defaults
    case (state)
        MAIN_GREEN : begin main_light = GREEN; side_light = RED; end
        ...
    endcase
end
```

*Twelve flip-flops: eight for the timer and FOUR for the state — Yosys re-encoded the 2-bit state to one-hot by itself. Predict, then check the log.*



## Binary or One-Hot — and How to Ask for It

**The state names in your code are symbols; the tool chooses the bit pattern.** Binary uses the fewest flip-flops; one-hot uses one flip-flop per state but makes the next-state and output logic trivially shallow — which on an FPGA, where flip-flops are free and logic depth is not, is usually the faster choice.

### Requesting an encoding with an attribute

```
(* fsm_encoding = "one-hot" *) reg [1:0] state, next;    // or "binary"

// Verified with Yosys 0.33 on a 4-state FSM, after abc -g AND,OR,XOR,NAND,NOR:
//   binary   : 9 cells, 2 flip-flops
//   one-hot   : 9 cells, 4 flip-flops    <-- more FFs, shallower next-state logic

// And Yosys chose one-hot BY ITSELF for traffic_fsm -- read the log:
//   FSM_RECODE: mapping auto encoding to `one-hot` for this FSM
```

### Always write a safe default

With one-hot,  $N$  states use  $2^N$  possible patterns — the vast majority illegal. A single upset bit puts the machine somewhere it can never leave. **default: next = IDLE;** costs nothing and guarantees recovery.

### A note from building this lab

On this 4-state machine the two encodings tie on cell count — the advantage of one-hot only appears when the state count grows and the next-state decode would otherwise get deep. **Do not assume; measure.** And read the log: the tool may already have re-encoded your FSM without being asked, which is exactly what happened to traffic\_fsm above.



## Where the Output Comes From Changes the Timing

**Moore outputs depend on the state alone; Mealy outputs depend on the state AND the current inputs.** A Mealy machine reacts one cycle sooner and often needs fewer states — but its output is combinational from an input, so it can glitch and it lands on the wrong side of a timing path.

|                         | Moore                               | Mealy                                   |
|-------------------------|-------------------------------------|-----------------------------------------|
| Output is a function of | state only                          | state and inputs                        |
| Reacts to an input      | one clock later                     | in the same cycle                       |
| Output glitches?        | no — it is registered               | yes — it follows the input              |
| Typical state count     | more                                | fewer                                   |
| Safe to drive off-chip? | yes                                 | not without registering it              |
| Use it when             | you want clean, predictable outputs | you need the earliest possible response |

### Lab L3 — build the same 1011 sequence detector both ways

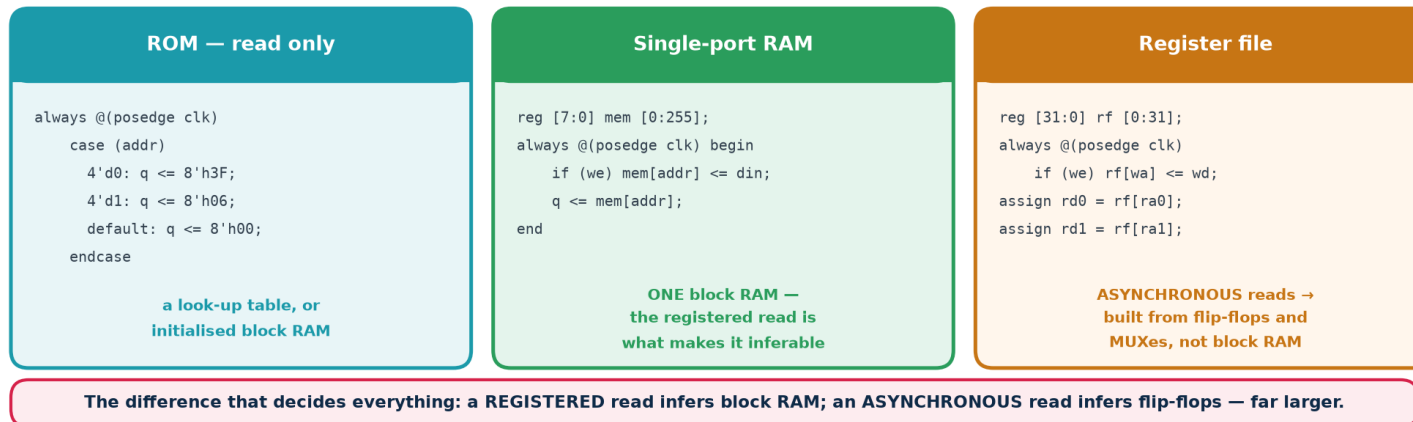
Topic4\_Lab/rtl/seq\_detect\_1011.v implements both in one module so you can watch the two **found** outputs on the same waveform. The Mealy output pulses on the clock edge where the final 1 arrives; the Moore output pulses one cycle later. Overlapping detection is handled by returning to the correct partial-match state rather than to IDLE — 1011011 contains two matches, not one.



# Inferring RAM — Write It the Way the Tool Expects

**You do not instantiate a RAM; you write a pattern the tool recognises.** Get the pattern right and you get a dedicated block RAM. Get it slightly wrong and you get thousands of flip-flops, or a design that will not fit at all.

Inferring memory — write the pattern the tool recognises



## Topic4\_Lab/rtl/sync\_ram.v — synchronous read

```
reg [W-1:0] mem [0:DEPTH-1];
always @(posedge clk) begin
  if (we) mem[waddr] <= wdata;
  rdata <= mem[raddr];      // REGISTERED read -- this is what infers block RAM
end
```

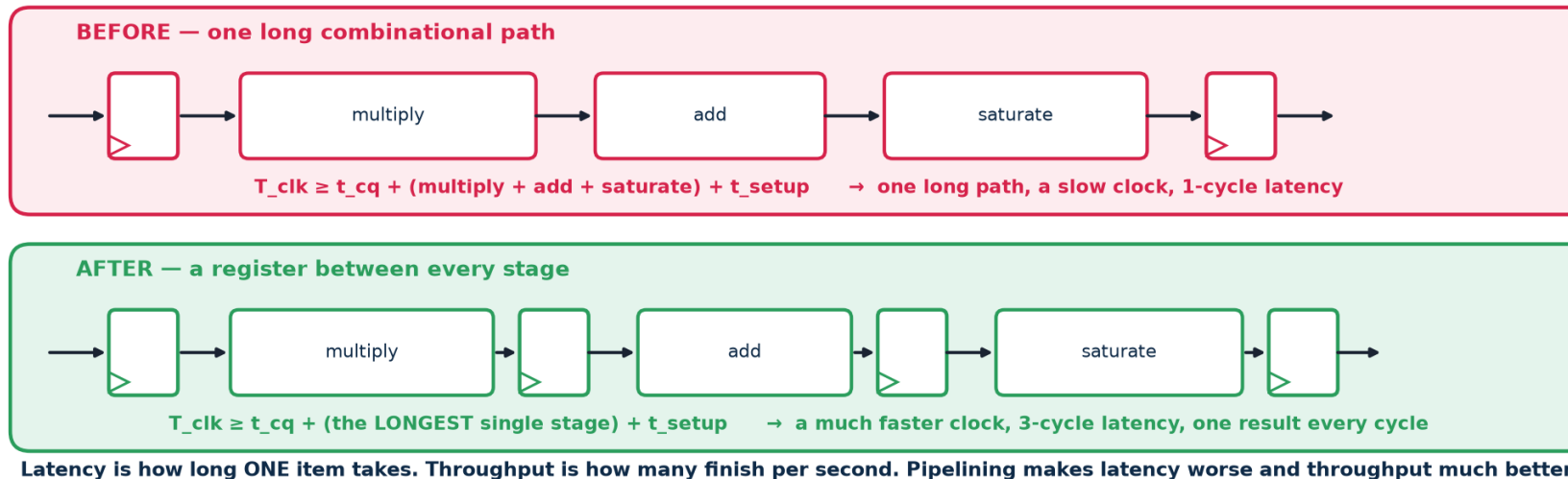
Write **assign rdata = mem[raddr];** instead — an asynchronous read — and no FPGA block RAM can implement it, so the tool builds the whole array out of registers and multiplexers. Same intent, wildly different silicon.



# Pipelining — Trading Latency for Throughput

**The clock period must be longer than the slowest combinational path between two flip-flops.** If that path is too slow, cut it in half with a register in the middle. You now need two cycles to get an answer — but you can start a new one every cycle.

## Pipelining — trade latency for throughput by inserting registers



### What you gain and what you pay

- + Throughput: one result per clock, at a much higher clock.
- Latency: the first result arrives N cycles later.
- Area: the pipeline registers, and any control signal that must now be delayed to stay aligned with its data.

### The trap

**Every signal travelling alongside the data must be delayed by the SAME number of stages.** Pipeline the datapath and forget the valid bit, and your results arrive correctly but are marked valid two cycles early. Carry valid, and any tag or address, through the identical register chain.





# Predict the Hardware — Eight Snippets

Show each snippet, ask 'what does the tool build?', then run it. Full answers and the synthesis reports are in workbook section T4-B.

| # | Snippet                                     | What does synthesis produce?           |
|---|---------------------------------------------|----------------------------------------|
| 1 | assign y = sel ? b : a;                     | One W-bit 2:1 multiplexer, no state    |
| 2 | always @(*) if (en) y = d;                  | A transparent latch on y               |
| 3 | always @(posedge clk) q1 = d; q2 = q1;      | ONE flip-flop (verified above)         |
| 4 | always @(posedge clk) q <= {q[6:0], sin};   | An 8-bit shift register                |
| 5 | for (i=0;i<8;i=i+1) s = s + d[i];           | An adder tree — eight adders, unrolled |
| 6 | assign z = ~ result;                        | A NOR reduction tree — the zero flag   |
| 7 | rdata <= mem[raddr]; inside a clocked block | A block RAM with registered read       |
| 8 | case (sel) with an x in sel                 | Simulation/synthesis mismatch — banned |

## The habit this checkpoint is building

**Predict, then verify.** A designer who can look at a page of RTL and describe the netlist before running the tool will catch problems in review, in minutes. A designer who cannot will find them in the lab, in days — or on the silicon, in months.

## SUBTOPIC 4C

# Writing RTL Code for Basic Digital Circuits

Twenty-two designs, five self-checking testbenches, one capstone. Everything from here on exists as runnable code.

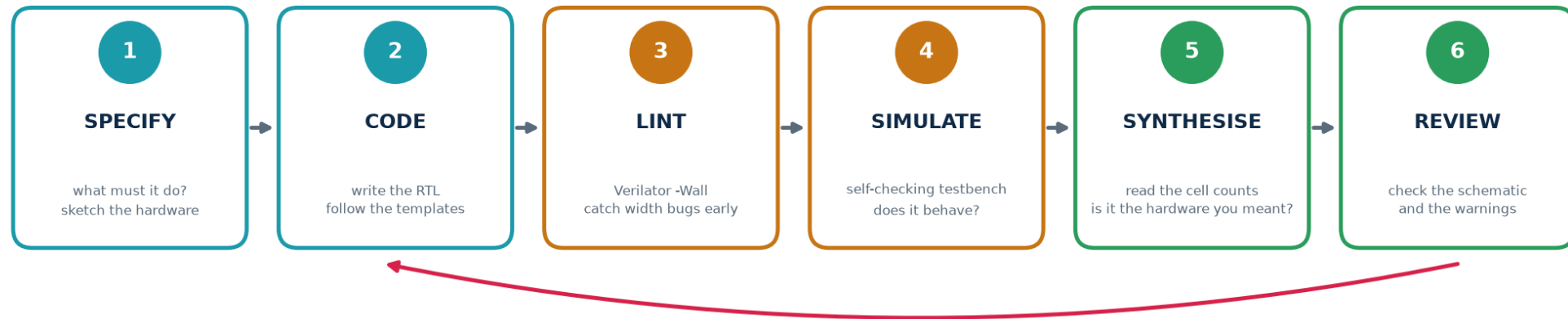
- The RTL development flow — specify, code, lint, simulate, synthesise
- A catalogue of the standard blocks and how each is written
- Verification: testbench anatomy, self-checking, scoreboards, randomisation
- Two full designs: a synchronous FIFO and an 8N1 UART
- Two real bugs, found while building this lab, and how they were caught



# The Loop You Will Live In

**Every RTL block goes round this loop, and the order is not negotiable.** Lint takes one second and catches width bugs; simulation takes a minute and catches logic bugs; synthesis takes longer and catches structural bugs. Run them cheapest first.

## The RTL design loop — you will go round this many times per day



**anything wrong at ANY step sends you back to the code**

*Steps 3 and 5 are the ones beginners skip — and they are the two that tell you whether you wrote **HARDWARE** or just wrote code that compiles.*

### The order matters more than the tools

**Students who skip lint and go straight to the waveform viewer spend hours chasing symptoms of a truncation Verilator would have named in one second. Students who skip synthesis until the end discover their beautiful, fully-verified design contains forty latches.** Lint, simulate, synthesise — every time, in that order.



# The Standard Blocks — and the Idiom That Builds Each One

**Almost all digital design is assembling these.** Every row is a real, verified file in Topic4\_Lab/rtl/.

| Block                           | File                                    | Core idiom                                           | Lab |
|---------------------------------|-----------------------------------------|------------------------------------------------------|-----|
| <b>2:1 / 4:1 multiplexer</b>    | <code>mux2.v</code> <code>mux4.v</code> | <code>assign y = sel ? b : a; / case</code>          | L1  |
| <b>3-to-8 decoder</b>           | <code>decoder3to8.v</code>              | <code>y = 0; if (en) y[sel] = 1'b1;</code>           | L1  |
| <b>8-input priority encoder</b> | <code>priority_encoder8.v</code>        | <code>casez</code> with ? wildcards                  | L1  |
| <b>8-operation ALU + flags</b>  | <code>alu.v</code>                      | <code>defaults + case; {1'b0,a}+{1'b0,b}</code>      | L1  |
| <b>7-segment decoder</b>        | <code>seven_seg.v</code>                | <code>case</code> on the nibble, active-low output   | L1  |
| <b>N-bit ripple adder</b>       | <code>adder_gen.v</code>                | <code>generate for + full_adder</code> instances     | L1  |
| <b>Register with enable</b>     | <code>reg_en.v</code>                   | <code>if (!rst_n) ... else if (en) q &lt;= d;</code> | L2  |
| <b>Shift register</b>           | <code>shift_reg.v</code>                | <code>q &lt;= {q[W-2:0], sin};</code>                | L2  |
| <b>Up/down counter</b>          | <code>counter.v</code>                  | <code>q &lt;= q + 1'b1;</code> with wrap and load    | L2  |
| <b>Edge detector</b>            | <code>edge_detect.v</code>              | <code>sig &amp; ~sig_d</code>                        | L2  |
| <b>CDC synchroniser</b>         | <code>synchroniser.v</code>             | <code>sync &lt;= {sync[0], async_in};</code>         | L2  |
| <b>Switch debouncer</b>         | <code>debouncer.v</code>                | <code>counter + stable-for-N-cycles</code>           | L2  |
| <b>Clock divider / tick gen</b> | <code>clk_divider.v</code>              | <code>counter + terminal-count pulse</code>          | L2  |



## ...and the Bigger Blocks

**These are the designs where architecture starts to matter.** Each is a small system: a controller, a datapath, and a decision about how they communicate.

| Block                    | File              | Core idiom                                    | Lab |
|--------------------------|-------------------|-----------------------------------------------|-----|
| Traffic-light controller | traffic_fsm.v     | three-block Moore FSM + dwell timer           | L3  |
| Vending machine          | vending_fsm.v     | FSM with a credit datapath and change         | L3  |
| 1011 sequence detector   | seq_detect_1011.v | Moore AND Mealy in one module, overlapping    | L3  |
| Synchronous FIFO         | sync_fifo.v       | dual pointers with an extra MSB; inferred RAM | L4  |
| Synchronous RAM          | sync_ram.v        | mem[waddr] <= wdata; rdata <= mem[raddr];     | L4  |
| UART transmitter         | uart_tx.v         | FSMD: shift register + bit timer + FSM        | L5  |
| UART receiver            | uart_rx.v         | half-bit alignment, mid-bit sampling, CDC     | L5  |

### Verified synthesis results — you should be able to predict these

**counter (4-bit): 38 cells, 4 FF · traffic\_fsm: 52 cells, 12 FF · uart\_tx: 120 cells, 27 FF · uart\_rx: 120 cells, 36 FF · sync\_fifo (8×8): 255 cells, 72 FF**

Before you run the tool on your own design, write down your prediction for the flip-flop count. Count the bits of state you declared. If the tool disagrees with you by more than a few, one of you is wrong — and it is worth finding out which.



# A Display Decoder and a Generated Adder

## seven\_seg.v — a lookup table in case form

```
always @(*) begin
    seg = 7'b111_1111;    // all off
    case (nibble)
        4'h0: seg = 7'b100_0000;
        4'h1: seg = 7'b111_1001;
        ...
        default: seg = 7'b011_1111; // '-'
    endcase
end
```

Active-LOW because the display's common anode is tied high — read the board schematic, not the datasheet's picture.

## adder\_gen.v — structure from a loop

```
genvar i;
assign c[0] = cin;
generate
    for (i=0; i<W; i=i+1) begin: bit_slice
        full_adder u_fa (.a(a[i]), .b(b[i]),
            .cin(c[i]), .sum(sum[i]),
            .cout(c[i+1]));
    end
endgenerate
```

W full-adder instances chained by the carry. Slow for large W — the carry ripples — which is exactly the point of the experiment.

## Experiment for Lab L1

Synthesise adder\_gen at W = 4, 16 and 64 and watch the longest path grow linearly with W. Then look up carry-lookahead and carry-select adders and ask why a synthesiser given **assign sum = a + b;** will usually produce something faster than your hand-written ripple chain. Lesson: describe intent, let the tool choose the structure — unless you have a specific reason not to.

## TOPIC 4C · VERIFICATION

# Simulating and Verifying the Functionality of RTL Designs

The syllabus names this as a practical outcome in its own right. Untested RTL is not a design; it is a guess.

- Testbench anatomy — the six parts every testbench has
- Self-checking: never read a waveform to decide if a test passed
- Reference models, scoreboards and randomised stimulus
- Waveform debugging, and the debug ladder to climb when stuck



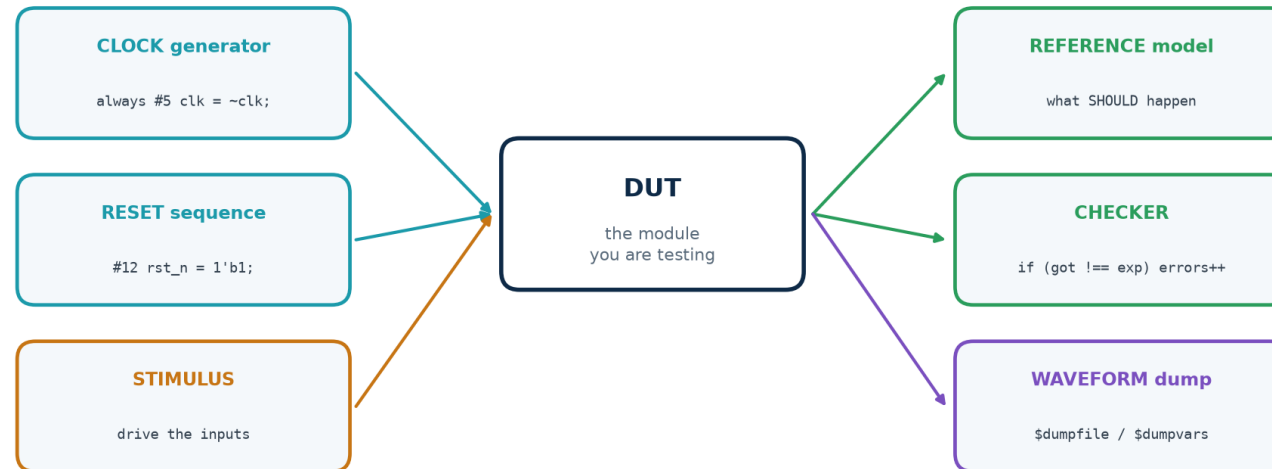
# Anatomy of a Testbench

**A testbench is a module with no ports.** It creates a clock, drives the design, and decides — automatically — whether the answers were right. It is NOT synthesised, so the whole language is available to you.

## Anatomy of a self-checking testbench

**A testbench you have to READ A WAVEFORM to grade is a testbench that will not catch a regression.**

*Waveforms are for DIAGNOSING a failure; the PASS/FAIL message is for DETECTING one. You need both, and they do different jobs.*



### The rule that separates a testbench from a demo

**A testbench must print PASS or FAIL and must be readable by a machine.** If deciding whether the run was correct requires a human to look at a waveform, you cannot run it in a regression, you cannot run it overnight, and you will stop running it entirely within a week.





# A Complete, Minimal, Self-Checking Testbench

## Everything a testbench needs, in 24 lines

```
`timescale 1ns / 1ps
module tb_mux2;
    reg [7:0] a, b; reg sel; wire [7:0] y;
    integer errors = 0;

    mux2 #(.W(8)) u_dut (.a(a), .b(b), .sel(sel), .y(y));    // 1. instantiate

    // 2. one place that decides pass or fail
    task check(input [7:0] got, input [7:0] exp, input [255:0] msg);
        if (got !== exp) begin
            $display("FAIL %0t: %0s got %h expected %h", $time, msg, got, exp);
            errors = errors + 1;
        end
    endtask

    initial begin
        $dumpfile("mux2.vcd"); $dumpvars(0, tb_mux2);    // 3. waveform dump
        a = 8'hA5; b = 8'h5A;
        sel = 1'b0; #1 check(y, 8'hA5, "sel=0 should pass a");    // 4. stimulus + check
        sel = 1'b1; #1 check(y, 8'h5A, "sel=1 should pass b");
        if (errors == 0) $display("PASS - mux2");    // 5. verdict
        else $display("FAIL - %0d errors", errors);    // 6. stop
        $finish;
    end
endmodule
```

Note **!==** not **!=** — the four-state comparison. With **!=**, comparing against an x gives x, which is not true, so the check silently passes. Always use **===** and **!==** in a testbench.



# Clock, Reset, and When to Drive and Sample

## The clock and reset boilerplate

```
// A 100 MHz clock: 10 ns period, so toggle every 5 ns.
localparam integer CLK = 10;
initial clk = 1'b0;
always #(CLK/2) clk = ~clk;

// Reset: assert before the first edge, release BETWEEN edges.
initial begin
    rst_n = 1'b0;
    repeat (3) @(posedge clk);
    #(CLK/5) rst_n = 1'b1;    // released just after an edge, not ON one
end
```

## A bug that cost real time while building this lab

**The FSM testbench originally released reset exactly ON a clock edge.** That is a race: the reset and the clock change at the same instant, so the state register saw an ambiguous condition and the state stayed x for two cycles. The sequence detector then found one match instead of two, and the design looked broken. It was not.

**Rule:** drive stimulus a small delay AFTER the active edge, and sample a small delay BEFORE the next one. Never change an input at the same instant as the clock.

## Timescale matters

``timescale 1ns / 1ps` means the unit of #1 is a nanosecond and the simulator rounds to a picosecond. Change the first number and every delay in the file changes meaning. Put the same timescale in every file, or put it in one included header.



# Reference Models, Scoreboards and Randomisation

**Hand-written expected values do not scale past about twenty cases.** Beyond that you need a model that computes the right answer independently, and random stimulus to reach the corners you would never think of.

## Topic4\_Lab/tb/tb\_mem.v (extract)

```
// A scoreboard: an independent model of what should be inside the FIFO.
reg [W-1:0] model [0:DEPTH-1];
integer      mhead, mtail;
function integer mcount; mcount = mtail - mhead; endfunction

task fifo_push(input [W-1:0] dat);
  begin
    wr_en = 1'b1; wr_data = dat; @(posedge clk); #1 wr_en = 1'b0;
    model[mtail % DEPTH] = dat; mtail = mtail + 1; // model, not DUT
  end
endtask

// Randomised, but SEEDED -- so a failure reproduces exactly.
for (k = 0; k < 200; k = k + 1) begin
  if (($random(seed) % 2) == 0) begin if (!full) fifo_push($random(seed)); end
  else begin if (!empty) fifo_pop(); end
  check1(count === mcount(), "fifo count disagrees with the scoreboard");
end
```

**The model must be written from the SPECIFICATION, never by reading the RTL.** A model derived from the design under test will happily agree with the design's bugs.



# When It Does Not Work — Climb the Ladder

**Debugging RTL is not guessing; it is a procedure.** Work from the cheapest check to the most expensive, and always find the **FIRST** moment the design went wrong, never the moment you noticed.

## Debugging RTL — work down this ladder, in this order

### It does not **COMPILE**

Read the **FIRST** error only — later ones are usually cascade damage.  
Missing semicolon, missing endmodule, a reg driven by assign.

### It compiles but the output is **X**

Find the **FIRST** signal that goes x and work backwards. Usual causes:  
no reset, an unconnected port, a multi-driver, or reading past the end of a vector.

### It simulates but gives the **WRONG** answer

Add \$display at the point of divergence. Compare against a reference model.  
Check blocking vs non-blocking, and check your width arithmetic.

### It simulates right but **SYNTHESISES** wrong

You used something outside the synthesisable subset, or you inferred a latch.  
Read every synthesis warning. Compare the cell count with what you expected.

### It synthesises but **FAILS TIMING**

That is Topic 6. For now: shorten the critical path, or pipeline it.

### The single most effective habit

**Simulate after every few lines. A bug found in ten lines of new code takes a minute; the same bug found in three hundred takes an afternoon.**

## Chasing an x

x is contagious — by the time you see it on your output it may have travelled through ten signals. In GTKWave, add the suspect signal, find the **FIRST** edge where it goes x, then add whatever drives it and repeat. You are looking for the source, not the symptom.

## Reading a VCD

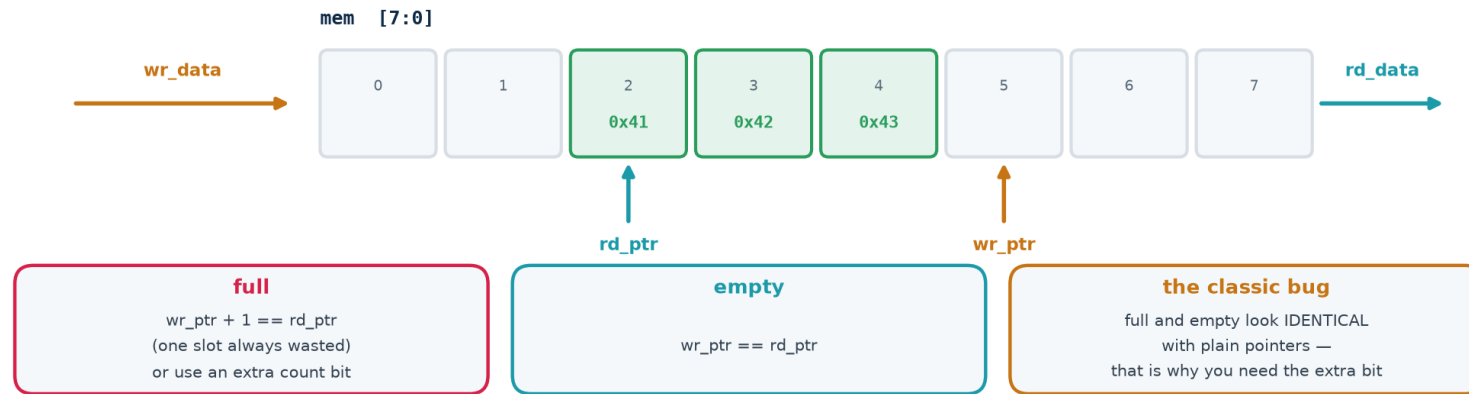
`$dumpvars(0, tb);` dumps every signal at every level — start there.  
`$dumpvars(1, tb.u_dut);` dumps one level of one instance, for when the file gets too big to open.



# A Synchronous FIFO — Pointers, and the Extra Bit

**A FIFO is a memory plus two pointers.** The only hard part is telling FULL from EMPTY: in both cases the two pointers are equal. The classic solution is to give each pointer one extra bit.

A synchronous FIFO — the classic 'put it all together' design



## Topic4\_Lab/rtl/sync\_fifo.v — verified 255 cells, 72 FF at 8x8

```
reg [AW:0] wr_ptr, rd_ptr;           // AW+1 bits for a 2**AW-deep FIFO
assign empty = (wr_ptr == rd_ptr);   // identical, MSB included
assign full  = (wr_ptr[AW] != rd_ptr[AW]) && // wrapped a different number
               (wr_ptr[AW-1:0] == rd_ptr[AW-1:0]); // of times, same position
assign count = wr_ptr - rd_ptr;       // subtraction wraps correctly, for free
```

**Guard the pointers, not the caller:**  $wr\_en \ \& \ \sim full$  and  $rd\_en \ \& \ \sim empty$ . A FIFO that silently corrupts itself when someone writes to it while full is a system-level bug waiting to happen.

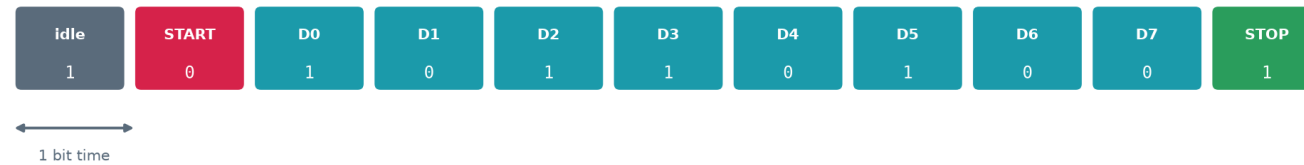


# A UART — the 8N1 Frame

**The UART is the capstone because it forces every idea in this topic together.** There is no clock on the wire: both ends agree a bit rate in advance and the receiver recovers timing from the start bit. That is the whole design problem.

## A real design target — the UART frame

one character = 10 bit-times, LSB first, no parity (8N1)



at 115 200 baud with a 50 MHz clock:  $50\,000\,000 / 115\,200 = 434$  clocks per bit

### Transmitter — an FSM plus a shift register

IDLE → START → DATA (×8) → STOP → IDLE,  
clocked by a baud-rate tick from a counter.

### Receiver — the same, plus one trick

Sample each bit in its MIDDLE, not at its edge —  
wait half a bit time after detecting the start bit.

## 8N1, and how you count clocks

Idle high · one START bit low · eight DATA bits, LSB first · no parity · one STOP bit high. Ten bit times per byte.

**CLKS\_PER\_BIT =  $f_{\text{clk}} / \text{baud}$**

50 MHz / 115200 = 434.03 → 434. The 0.03 error accumulates to 0.3 of a bit over ten bits: comfortably inside tolerance.

## The receiver's one trick

**Sample each bit in its MIDDLE, not at its edge.** On seeing the start bit fall, wait HALF a bit time and re-check the line is still low — that rejects a glitch. From then on, sample every FULL bit time and you land in the centre of every bit, with maximum margin against clock mismatch.



# The Receiver — an FSMD in Four States

## Topic4\_Lab/rtl/uart\_rx.v — verified 120 cells, 36 FF

```
synchroniser #(.STAGES(2)) u_sync (.clk(clk), .rst_n(rst_n),
                                   .async_in(rx), .sync_out(rx_sync)); // CDC first!

case (state)
  IDLE : begin
    clk_cnt <= {CW{1'b0}}; bit_idx <= 3'd0;
    if (!rx_sync) state <= START;           // falling edge seen
  end
  START: if (clk_cnt == HALF_BIT[CW-1:0]) begin
    if (!rx_sync) begin clk_cnt <= {CW{1'b0}}; state <= DATA; end // genuine
    else state <= IDLE;                                           // glitch
  end else clk_cnt <= clk_cnt + 1'b1;
  DATA : if (clk_cnt == FULL_BIT[CW-1:0]) begin
    clk_cnt <= {CW{1'b0}};
    shifter <= {rx_sync, shifter[7:1]};           // LSB arrives first
    if (bit_idx == 3'd7) state <= STOP; else bit_idx <= bit_idx + 1'b1;
  end else clk_cnt <= clk_cnt + 1'b1;
  STOP : if (clk_cnt == FULL_BIT[CW-1:0]) begin
    rx_data <= shifter; rx_valid <= 1'b1;
    frame_error <= ~rx_sync;                     // stop bit must be high
    state <= IDLE;
  end else clk_cnt <= clk_cnt + 1'b1;
endcase
```

*Three separate registers hold the state: the FSM state (2 bits), the clock counter (CW bits) and the bit index (3 bits). Counting those, plus the shifter and outputs, is how you predict 36 flip-flops before you run the tool.*



# The Bug That Made Some Bytes Work and Others Not

**This is a genuine bug found while writing this lab, kept because it is better teaching material than anything invented.** Read the symptom first and try to name the cause before turning to the next line.

## SYMPTOM

The loopback test passed for 0x00 and 0x55 but failed for 0xFF — which came back as 0xF7. One bit, in one pattern. Everything else about the design looked correct, and the transmitter was provably fine.

## CAUSE

The bit-timing limits were written `CLKS_PER_BIT[CW-1:0]`. The testbench used `CLKS_PER_BIT = 16` to keep runs short, so `CW = $clog2(16) = 4` — and 16 does not fit in 4 bits. It truncated to ZERO. `HALF_BIT` became 0, so the receiver never waited half a bit and sampled on bit BOUNDARIES instead of bit centres.

## WHY SOME BYTES

Sampling on a boundary gives whichever value wins the race — which only matters when adjacent bits DIFFER. 0x00 and 0xFF-with-a-low-start-bit differ in different places, so some patterns survived and others did not. Intermittent, data-dependent failure: the worst kind.

## FIX

Compute the limits as integers — localparam integer `FULL_BIT = CLKS_PER_BIT - 1;` — and slice them where they are USED. Verified at `CLKS_PER_BIT = 16, 27 and 434`. Verilator -Wall is clean on this form and flagged the alternatives.

### How it was actually caught, and what would have caught it sooner

It was caught by the loopback testbench, which compares the received byte with the byte that was sent — in one second, with no waveform. It would have been caught EARLIER by `verilator --lint-only -Wall`, which reports WIDTHTRUNC on exactly that expression. Lint costs a second; this bug cost an hour.





## What Those Two Bugs Actually Teach

### Bug 1 — width truncation in the UART

- A parameter used as an INDEX RANGE is not the same as a parameter used as a VALUE. Slicing a value that does not fit destroys it silently.
- **Test at more than one parameter value.** At 434 the bug is invisible, because 434 needs 9 bits and CW is 9. Only the short test value exposed it.
- **Verilator would have said WIDTHTRUNC. Lint first, always.**

### Bug 2 — state timing in the vending machine

- The credit register was cleared as the FSM ENTERED the DISPENSE state, so the output logic — which needs the credit to compute change — saw zero.
- **The fix is to clear it as the machine LEAVES that state.**
- **Lesson: in a Moore machine the outputs are computed FROM the state you are in. Anything that state's outputs depend on must still be valid while you are in it.**

### The meta-lesson worth stating out loud to the class

**Both bugs were found by a self-checking testbench, in seconds, and both would have been nearly impossible to find by looking at waveforms alone.** Neither was a syntax error; both compiled and simulated cleanly. This is what verification is FOR.

# Software Tools — Installation and Flow

The syllabus specifies Vivado Design Suite and ModelSim. The open-source chain does the same jobs and runs on any laptop.

- ❑ What each tool does, and which one to reach for first
- ❑ Installing the open-source chain — Linux, WSL2, macOS
- ❑ Installing and driving Vivado Design Suite
- ❑ Installing and driving ModelSim / Questa
- ❑ A one-page command reference to keep beside the keyboard



# Four Jobs, Several Tools for Each

**Every flow does the same four jobs.** Only the command names change. Learn the jobs and moving between vendors becomes a morning's work rather than a new skill.

## Three toolchains — the syllabus tools, and a free one for home

| VIVADO                                      | MODELSIM / QUESTA                           | OPEN-SOURCE                                    |
|---------------------------------------------|---------------------------------------------|------------------------------------------------|
| AMD / Xilinx · free WebPACK edition         | Siemens EDA · free Intel Starter edition    | Icarus + GTKWave + Yosys + Verilator           |
| simulate<br>built-in XSim                   | simulate<br>the industry reference          | simulate<br>Icarus Verilog, Verilator          |
| synthesise<br>yes, to real FPGA             | synthesise<br>no — simulation only          | synthesise<br>Yosys                            |
| implement<br>place & route, bitstream       | implement<br>no                             | implement<br>nextpnr, for some parts           |
| timing<br>full STA                          | timing<br>no                                | timing<br>basic, via OpenSTA                   |
| schematic<br>RTL and technology views       | schematic<br>no, but excellent waveforms    | schematic<br>Yosys show + graphviz             |
| size<br>~50 GB                              | size<br>~1-5 GB                             | size<br>~200 MB                                |
| in the syllabus<br>YES — the specified tool | in the syllabus<br>YES — the specified tool | in the syllabus<br>no — but ideal for practice |

Learn the CONCEPTS on the free tools at home, then apply them on Vivado and ModelSim in the lab. The flow and the vocabulary are identical.

| Job                 | Open-source                   | Vendor (syllabus)                | What it tells you                                   |
|---------------------|-------------------------------|----------------------------------|-----------------------------------------------------|
| Lint / static check | Verilator --lint-only         | Vivado report_methodology        | Width bugs, latches, unused signals — in one second |
| Simulate            | Icarus Verilog (iverilog/vvp) | ModelSim, Questa, Vivado xsim    | Does it behave correctly?                           |
| View waveforms      | GTKWave                       | ModelSim wave window, Vivado sim | Where and when it went wrong                        |
| Synthesise          | Yosys                         | Vivado synth_design              | What hardware it becomes; area; latch warnings      |



# The Open-Source Toolchain — Every Command

## Copy-paste installation

```
# ---- Ubuntu / Debian / WSL2 -----
sudo apt update
sudo apt install -y iverilog gtkwave verilator yosys graphviz

# ---- macOS (Homebrew) -----
brew install icarus-verilog gtkwave verilator yosys graphviz

# ---- Windows -----
# Best: install WSL2 (wsl --install in an admin PowerShell), then use
#       the Ubuntu commands above inside it.
# Or:   download the OSS CAD Suite release for Windows, unzip it, and
#       run its start.bat -- it contains all four tools, already built.

# ---- verify the installation -----
iverilog -V | head -1      # Icarus Verilog version 12.0
verilator --version        # Verilator 5.020
yosys -V                  # Yosys 0.33
gtkwave --version
```

## GTKWave on WSL2

GTKWave is a GUI. On Windows 11, WSLg runs Linux GUI applications with no extra setup — just type **gtkwave dump.vcd**. On Windows 10 you need an X server (VcXsrv) and **export DISPLAY=:0**, or simply install the native Windows GTKWave and open the .vcd file the WSL side wrote.



# Driving the Open-Source Chain — Four Commands

## Topic4\_Lab — these exact commands were used to verify this material

```
# 1. LINT -- one second, run it before anything else
verilator --lint-only -Wall -Wno-DECLFILENAME rtl/*.v

# 2. COMPILE + SIMULATE
iverilog -g2005 -Wall -o build/sim.vvp rtl/uart_tx.v rtl/uart_rx.v \
        rtl/synchroniser.v tb/tb_uart.v
vvp build/sim.vvp                # prints PASS or FAIL

# 3. VIEW WAVEFORMS (the testbench must call $dumpfile/$dumpvars)
gtkwave uart.vcd &

# 4. SYNTHESISE and look for latches
yosys -p "read_verilog rtl/counter.v; synth -top counter; stat"
yosys -p "read_verilog rtl/counter.v; synth -top counter" 2>&1 | grep -i latch

# ... or just use the scripts, which do all of the above for every lab
./scripts/lint.sh    && ./scripts/run_all.sh    && ./scripts/synth_all.sh
```

**Expected output:** LINT CLEAN · ALL LABS PASSED · ALL DESIGNS SYNTHESISED, NO LATCHES INFERRED



# Vivado Design Suite — Installation

**Vivado is the tool named in the syllabus.** The free ML Standard edition covers every device you will use on a training board. Budget an afternoon: the download is large and the installer is slow.

- 1. Create a free AMD/Xilinx account, then download the **Unified Installer** for Windows or Linux from the AMD downloads page.
- 2. Run it and choose **Vivado → Vivado ML Standard** (free, no licence file needed).
- 3. On the device page select ONLY the family your board uses — **Artix-7** for Basys 3 / Arty A7, **Zynq-7000** for Zybo / PYNQ. Selecting everything costs well over 100 GB.
- 4. Keep **Vivado Simulator (xsim)** and, on Windows, the **Cable Drivers** — without the drivers the board will not be detected.
- 5. Disk: allow 60–100 GB for one device family. RAM: 8 GB minimum, 16 GB comfortable.
- 6. Linux only: run `sudo <install>/data/xicom/cable_drivers/lin64/install_script/install_drivers/install_drivers` after installation.
- 7. Verify: `vivado -version` (source `settings64.sh` first on Linux).

## Boards commonly used for this course

**Basys 3** — Artix-7 xc7a35tcpg236-1. Switches, LEDs, four 7-segment digits. Ideal for L1–L3.

**Arty A7 / Nexys A7** — Artix-7, with a USB-UART bridge, so L5 talks to a terminal on the PC.

**Zybo / PYNQ-Z2** — Zynq-7000. More than this topic needs, but the same flow.

## Installation problems you will meet

- Installer hangs at 'Generating installed device list' — it is not hung, it takes several minutes.
- Board not detected — the cable drivers were skipped, or on Linux the udev rules were not installed.
- 'No such part' from a TCL script — the part string does not match your board. Edit it in `vivado_synth.tcl`.

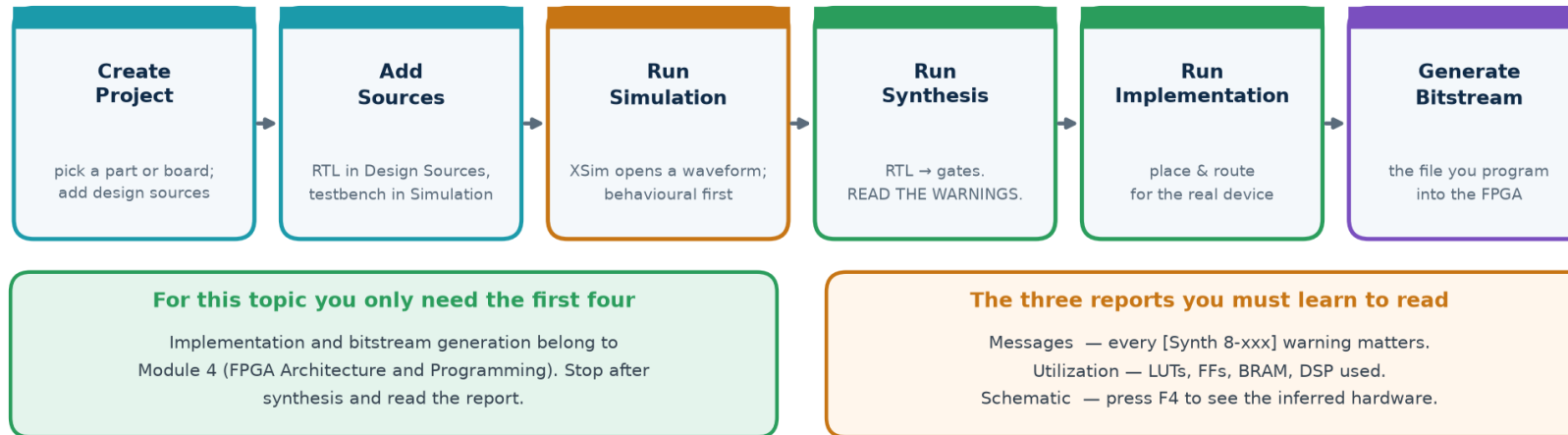
## Plan the lab session around this

Do not spend a scheduled lab hour installing Vivado. Either pre-install it on the machines, or set the download running at the start of the session and teach the open-source flow — which installs in under a minute — while it runs. The concepts are identical, and the students are learning RTL, not an installer.



# The Vivado Flow — GUI and Batch

## The Vivado project flow — what each stage actually does



### Topic4\_Lab/scripts/vivado\_sim.tcl and vivado\_synth.tcl

```
# Batch simulation -- three tools, always in this order
xvlog rtl/uart_tx.v rtl/uart_rx.v rtl/synchroniser.v tb/tb_uart.v # analyse
xelab -debug typical tb_uart -s tb_uart_sim                    # elaborate
xsim tb_uart_sim -runall                                       # run

# Batch synthesis with the provided script
vivado -mode batch -source scripts/vivado_synth.tcl -tclargs uart_tx
```

In the GUI the same steps are Create Project → Add Sources → Run Simulation → Run Synthesis → Open Synthesized Design. Learn the batch commands anyway: they are what you put in a makefile, and they are what a regression runs.



# What to Read After Synthesis — Not the Schematic

## scripts/vivado\_synth.tcl — change the part to match your board

```
create_clock -period 20.000 -name clk [get_ports clk] # 50 MHz, or timing means nothing
synth_design -top uart_tx -part xc7a35tcpg236-1
report_utilization -file build/vivado/uart_tx_utilization.rpt
report_timing_summary -file build/vivado/uart_tx_timing.rpt
```

### Utilisation report

LUTs, flip-flops, block RAMs, DSPs. Compare the flip-flop count with your own prediction from the source — a large disagreement means the tool built something you did not intend.

### Timing summary

Worst negative slack (WNS). Positive means the design meets the clock you constrained; negative means it does not, and the number tells you by how much. Without a create\_clock the report is meaningless.

### Messages tab

Every **[Synth 8-xxx]** warning. Read all of them once. Latch inference, width mismatch and unconnected port warnings all live here.

### Honesty note for the trainer

The Vivado and ModelSim scripts in Topic4\_Lab are working templates written from standard, version-stable commands, but they were NOT executed while this material was prepared — neither tool was installed in the authoring environment. Everything quoted from Icarus, Verilator and Yosys was really run. Check the vendor scripts against your installed version before the session; part numbers and menu names change between releases.





# ModelSim / Questa — Installation and Flow

## Installing

- ModelSim Intel FPGA Starter Edition is free and needs no licence; it ships with Intel Quartus Prime Lite. Questa Intel FPGA Starter is its successor — either is fine for this course.
- Windows: run the installer, accept the default path. Linux: it is 32-bit, so install the multilib packages **lib32z1 lib32ncurses6 libxft2:i386** first or vsim will not start.
- Verify with **vsim -version**.

## The three commands that matter

**vlib work** create the working library

**vlog rtl/\*.v tb/tb\_uart.v** compile into it

**vsim -voptargs=+acc work.tb\_uart** elaborate and load

**run -all** simulate to \$finish

*+acc keeps signal visibility — without it the optimiser removes the very signals you want to look at.*

## Topic4\_Lab/scripts/modelsim\_run.do — covers all five labs

```
vsim -c -gLAB=L3_fsm -do scripts/modelsim_run.do    # batch, prints PASS/FAIL
vsim      -do scripts/modelsim_run.do              # GUI, with waveforms added
```



# One-Page Command Card

Print this and keep it beside the keyboard for the whole module.

| Task          | Open-source                         | Vivado                      | ModelSim                     |
|---------------|-------------------------------------|-----------------------------|------------------------------|
| Lint          | verilator --lint-only -Wall rtl/*.v | report_methodology          | vlog (warnings)              |
| Compile       | iverilog -g2005 -o sim.vvp <files>  | xvlog <files>               | vlog <files>                 |
| Elaborate     | (part of iverilog)                  | xelab -debug typical <top>  | (part of vsim)               |
| Run           | vvp sim.vvp                         | xsim <snapshot> -runall     | vsim -c work.<top>; run -all |
| Waves         | gtkwave dump.vcd                    | Vivado sim wave window      | add wave -r /*               |
| Synthesise    | yosys -p "...; synth -top X; stat"  | synth_design -top X -part P | (not a synthesis tool)       |
| Area report   | stat                                | report_utilization          | —                            |
| Timing report | (none — no timing model)            | report_timing_summary       | —                            |
| Find latches  | grep -i latch on the log            | Messages: [Synth 8-327]     | —                            |

## TOPIC 4 · PRACTICAL

# The Lab Programme

Mapped to the syllabus practical component: 40 hours of RTL Design and Implementation Labs, plus synthesis and timing.

- L1 combinational · L2 sequential · L3 state machines · L4 memory · L5 UART capstone
- Twenty-two designs, five self-checking testbenches, three toolchains
- Deliberately broken examples, so failures can be seen in real tool reports
- Assessment rubric and the common-error table



# The Five Labs and Their Hours

**These hours map onto the syllabus practical component.** Every design listed exists, compiles, simulates to PASS and synthesises with no inferred latches.

| Lab                         | Hours | Designs                                                                    | Testbench | What it teaches                                   |
|-----------------------------|-------|----------------------------------------------------------------------------|-----------|---------------------------------------------------|
| <b>L1 Combinational</b>     | 8     | mux2 mux4 decoder3to8 priority_encoder8 alu<br>seven_seg adder_gen         | tb_comb.v | assign vs always @(*), defaults, case, generate   |
| <b>L2 Sequential</b>        | 8     | reg_en shift_reg counter edge_detect synchroniser<br>debouncer clk_divider | tb_seq.v  | the clocked templates, <=, clock enables, CDC     |
| <b>L3 State machines</b>    | 8     | traffic_fsm vending_fsm seq_detect_1011                                    | tb_fsm.v  | three-block FSM, safe defaults, Moore vs Mealy    |
| <b>L4 Memory &amp; FIFO</b> | 8     | sync_fifo sync_ram                                                         | tb_mem.v  | memory inference, pointers, scoreboards, \$random |
| <b>L5 UART capstone</b>     | 8     | uart_tx uart_rx (+ synchroniser)                                           | tb_uart.v | a complete FSMD, loopback, independent decoder    |

*Forty hours in total, matching the syllabus allocation for RTL Design and Implementation Labs. The synthesis and timing exercises inside each lab feed the further 15 h Design Synthesis and 10 h Timing Analysis allocations.*

## What every lab produces

- A clean lint run. · A simulation transcript ending in PASS.
- A synthesis report with a cell and flip-flop count, and no inferred latches.
- Two sentences from the student saying what the design became in hardware.

## Running short of time?

**Do not cut L5.** The UART is where combinational logic, sequential logic, an FSM, a datapath, a clock-domain crossing and a self-checking testbench all appear in one design — it is the only exercise that shows students how the pieces fit. Cut depth in L1 instead; those designs are the easiest to catch up on alone.



# How to Run a Lab Session

## BEFORE

Students clone Topic4\_Lab and run `./scripts/lint.sh` and `./scripts/run_all.sh` unmodified. Everything must print PASS before anyone edits anything — that proves the environment works and removes 'it does not build' from the session.

## READ

Open one design and one testbench side by side. Ask the room to predict the flip-flop count from the source, write the predictions on the board, then run the synthesiser and compare. Ten minutes; it changes how they read code.

## BREAK

Introduce a fault deliberately: remove a default, change a `<=` to `=`, drop a case branch. Re-run lint, simulation and synthesis and see which one catches it. This is the single most effective exercise in the whole topic.

## BUILD

The exercise proper — extend or write a design from the workbook. Every exercise has a specification, a required interface and an expected testbench result, so the student always knows when they are finished.

## REPORT

Submit: source, the simulation transcript showing PASS, the synthesis utilisation figures, and two sentences on what the design became in hardware. The last item is the one being assessed.



# The Errors Your Students Will Actually Hit

**Nine out of ten support questions in this topic are one of these.** Put this table on the wall of the lab.

| Symptom                                    | Almost always means                                 | Fix                                               |
|--------------------------------------------|-----------------------------------------------------|---------------------------------------------------|
| <b>Output is x from time zero</b>          | A register was never reset, or a wire has no driver | Reset every register; check the port connections  |
| <b>Output is x after working</b>           | Two drivers, or reading past the end of a vector    | One signal, one driving block                     |
| <b>'Inferred latch for signal y'</b>       | A branch of always @(*) leaves y unassigned         | Default assignment at the top of the block        |
| <b>Shift register is one flip-flop</b>     | Blocking = used in a clocked block                  | Use <= in every always @(posedge clk)             |
| <b>Result wraps at the wrong value</b>     | Width truncation on an expression                   | Make the target one bit wider; run Verilator      |
| <b>Signal not found in GTKWave</b>         | \$dumpvars scope too narrow, or the file is stale   | \$dumpvars(0, tb); and re-run before re-opening   |
| <b>Testbench passes but hardware fails</b> | casex, full_case, or an initial block               | Remove all three; reset properly                  |
| <b>FSM is stuck in an unknown state</b>    | No default branch; reset released on an edge        | default: next = IDLE; release reset between edges |
| <b>Works at one parameter, not another</b> | A constant sliced to a width that cannot hold it    | Compute as integer; slice at the point of use     |



## What 'Done' Looks Like

**Assess the hardware understanding, not the typing.** A student who can explain what their code became has learned the topic; one whose simulation happens to pass may not have.

| Level     | Evidence                                                                                                                                                                         |
|-----------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Pass      | Design compiles, lint is clean, the supplied testbench prints PASS, and synthesis reports no inferred latches.                                                                   |
| Good      | All of the above, plus: the student predicted the flip-flop count correctly and can point to which lines produced them.                                                          |
| Strong    | All of the above, plus: a self-checking testbench of the student's own with a reference model, and a sensible explanation of the timing report.                                  |
| Excellent | All of the above, plus: the student found and diagnosed a real bug — theirs or a planted one — and can explain which stage of the flow should have caught it and why it did not. |

### One question to ask every student, every session

**"Point at a line and tell me what it became."** If the answer is fluent, they have understood RTL design. If it is 'it assigns y', go back to the inference map.



## Terms Used in This Topic — 1 of 2

| Term                        | Meaning                                                                                                  |
|-----------------------------|----------------------------------------------------------------------------------------------------------|
| <b>RTL</b>                  | Register-transfer level — describing what happens to data between clock edges, without specifying gates. |
| <b>Synthesis</b>            | Translating RTL into a gate-level netlist for a target technology.                                       |
| <b>Netlist</b>              | A list of gates and flip-flops and the wires between them — what synthesis outputs.                      |
| <b>Inference</b>            | The synthesiser recognising a code pattern and producing the corresponding hardware.                     |
| <b>Latch</b>                | Level-sensitive storage. Transparent while enabled. In RTL, almost always an accident.                   |
| <b>Flip-flop</b>            | Edge-triggered storage. Captures its input at one clock edge and holds it until the next.                |
| <b>Blocking (=)</b>         | Assignment that takes effect immediately, before the next statement. Combinational blocks only.          |
| <b>Non-blocking (&lt;=)</b> | Assignment scheduled to take effect at the end of the time step. Clocked blocks only.                    |
| <b>Sensitivity list</b>     | The events that make a procedural block run. @(*) for combinational, @(posedge clk) for clocked.         |
| <b>Net / variable</b>       | wire is a net, driven continuously; reg is a variable, assigned inside a procedural block.               |
| <b>Four-state logic</b>     | 0, 1, x (unknown) and z (high impedance).                                                                |
| <b>Elaboration</b>          | Building the design hierarchy and resolving parameters and generate blocks, before simulation.           |





## Terms Used in This Topic — 2 of 2

| Term                 | Meaning                                                                                                          |
|----------------------|------------------------------------------------------------------------------------------------------------------|
| <b>FSM</b>           | Finite state machine — a state register plus next-state and output logic.                                        |
| <b>Moore / Mealy</b> | Outputs from state alone / from state and inputs.                                                                |
| <b>One-hot</b>       | A state encoding using one flip-flop per state, exactly one of which is 1.                                       |
| <b>FSMD</b>          | Finite state machine with a datapath — a controller plus registers and arithmetic it steers.                     |
| <b>Metastability</b> | A flip-flop's output settling unpredictably after a setup or hold violation.                                     |
| <b>CDC</b>           | Clock domain crossing — moving a signal between two unrelated clocks.                                            |
| <b>Synchroniser</b>  | Two or more chained flip-flops that give metastability time to decay.                                            |
| <b>Pipelining</b>    | Inserting registers to shorten the longest combinational path, raising throughput at the cost of latency.        |
| <b>Slack / WNS</b>   | Time left over on a timing path; worst negative slack is the worst such path in the design.                      |
| <b>Testbench</b>     | A port-less module that instantiates the design, drives it, and checks the results automatically.                |
| <b>Scoreboard</b>    | An independent model of expected behaviour, written from the specification, that the testbench compares against. |
| <b>VCD</b>           | Value Change Dump — the waveform file format \$dumpvars writes and GTKWave reads.                                |



# The Ten Things That Matter

---

- Verilog describes hardware. Sketch the circuit first, then write the code that describes it.
- `reg` is not a register. What you assign it with, and in what block, decides the hardware.
- Three constructs: **`assign`**, **`always @(*)`**, **`always @(posedge clk)`**. Know what each becomes.
- `=` in combinational blocks, `<=` in clocked blocks. Never mix them for one variable.
- Assign every output on every path, or you get a latch. Default first, then override.
- Size every literal. Width truncation is silent, and it is the bug that comes back.
- Three-block FSMs, with a safe default that recovers from any illegal state.
- Anything crossing a clock domain goes through two flip-flops — and buses need Gray coding or a handshake.
- A testbench prints PASS or FAIL. If a human has to read a waveform to decide, it is not a testbench.
- Lint, then simulate, then synthesise — and read the synthesis log every single time.

## Where this leads

Topic 5 takes this RTL into synthesis proper — constraints, optimisation, and reading a timing report until it means something. Everything you build from here on is written in the language you have just learned; the difference is that the clock, the area and the power start to have opinions about it.

# RTL Design Using HDL

Deck · workbook · 22 verified designs · 5 self-checking testbenches · 3 toolchains.

- ▢ Slides: this deck, for delivery
- ▢ Workbook: Module2\_Topic4\_Tutorial\_Practice\_Workbook.docx — tutorials, exercises and full solutions
- ▢ Lab: Topic4\_Lab/ — rtl/, tb/, scripts/, all verified end to end
- ▢ Next: Module 2 Topic 5 — synthesis, constraints and timing closure